

DEEPSEEK HARNESS

# DeepSeek Harness 插件开发指南

从 Cordis 插件框架到 Profile、Bundle、工具、Skill、命令、系统提示词、Agent Preset 与客户端插件的完整开发路径，全部案例源自 @deepseek-ai/dsh 真实代码库。

版本 0.1.0-rc.6 · @deepseek-ai/cordis 4.0.1

# 目录

---

## DeepSeek Harness 插件开发指南

这本书讲什么

读者与前置条件

阅读路径

关于「真实案例」

## 第 1 章 体系总览

1.1 DeepSeek Harness 是什么

1.2 三层架构

1.3 插件有几种「形态」

1.4 一次启动的完整数据流

1.5 四个贯穿全书的核心理念

1.6 关键术语表

## 第 2 章 Cordis 框架核心

2.1 Context：一切的起点

2.2 插件的三种写法

2.3 服务：provide / inject / Service

2.4 effect：资源所有权与清理

2.5 事件总线

2.6 生命周期：Fiber 的状态机

2.7 配置校验：Schemastery + 标准 Schema

2.8 作用域：extend / isolate / intercept

2.9 第一个可运行插件：把它们串起来

2.10 本章小结

## 第 3 章 Profile、Bundle 与 patch 层

3.1 目录结构与三份文件

3.2 三层 patch 的叠加顺序

3.3 patch 条目的语义

3.4 bundle：dsh.bundle.patch 声明

3.5 模块解析：两锚点

3.6 实战：创建并挂载自己的 bundle

3.7 常见陷阱

3.8 本章小结

## 第 4 章 服务与依赖注入

4.1 服务是 DSH 的「毛细血管」

4.2 提供服务：ctx.provide 与 Service

4.3 消费服务：inject（硬）与 ctx.get（软）

4.4 作用域与隔离：isolate / intercept / extend

4.5 实战：一个计数器服务 + 消费者（可运行）

4.6 服务的生命周期耦合（重点理解）

4.7 常见陷阱

4.8 本章小结

## 第 5 章 工具开发

5.1 工具注册表：ToolRuntime (ctx.tools)

5.2 defineTool：类型化工具定义

5.3 执行上下文 exec 与取消

5.4 执行流水线（扩展点全景）

5.5 并发：isConcurrencySafe

5.6 工具拥有的 UI 呈现：presentCall / presentResult

5.7 案例一：复刻 todo\_write（最完整的参考实现）

5.8 案例二：一个带 HTTP 请求的工具

5.9 案例三：只读 + 可并发 + 带 read 视图的工具

5.10 工具开发检查清单

5.11 模型体验与 KV Cache 影响

5.12 本章小结

## 第 6 章 Skill 开发

6.1 Skill 是什么，如何进入模型

6.2 SKILL.md 格式

6.3 发现根目录与 rank

6.4 写一个 Skill（实战）

6.5 自定义 Skill 提供方（进阶）

6.6 目录变更检测（了解即可）

6.7 Skill vs 工具：什么时候用哪个

6.8 本章小结

## 第 7 章 命令开发

- 7.1 命令注册表: `ctx.commands`
- 7.2 `CommandResult`
- 7.3 案例: 复刻 `/goal`
- 7.4 案例: 一个自定义命令 `/ls`
- 7.5 命令 vs 工具 vs Skill
- 7.6 本章小结

## 第 8 章 系统提示词

- 8.1 组装注册表: `ctx.systemPrompt`
- 8.2 四种扩展方式
- 8.3 案例一: 注入部署级 persona
- 8.4 案例二: 动态上下文 (注入当前环境)
- 8.5 案例三: complete 段——接管整个提示词
- 8.6 模型体验与 KV Cache
- 8.7 常见陷阱
- 8.8 本章小结

## 第 9 章 Agent Preset 与作用域组合

- 9.1 两个平面: `host-plane` 与 `agent-plane`
- 9.2 目录结构与两份文件
- 9.3 `agent.cordis.yml` 的写法
- 9.4 实战: 复制 `standard` 做定制 preset
- 9.5 挂载与 `join`: 会话如何「使用」一个 preset
- 9.6 常见陷阱
- 9.7 本章小结

## 第 10 章 客户端插件

- 10.1 双面包: 一个包, 两半代码
- 10.2 `dsh.client manifest`
- 10.3 客户端 bundle 与加载
- 10.4 客户端插件的 `apply(ctx)`
- 10.5 Slot 系统: 客户端插件的扩展点
- 10.6 一个最小客户端插件 (骨架)
- 10.7 宿主插件 vs 客户端插件: 何时用哪个
- 10.8 本章小结

## 第 11 章 安全、沙箱与审批

- 11.1 三件套总览
- 11.2 沙箱模式
- 11.3 审批: ctx.approval
- 11.4 凭据: ctx.credentials
- 11.5 文件系统服务: ctx.fs
- 11.6 权限预设: 一个动作切三档
- 11.7 常见陷阱
- 11.8 本章小结

## 第 12 章 调试、热更新与发布

- 12.1 检查配置树: --dump-config
- 12.2 诊断「插件没生效」
- 12.3 热更新 (HMR)
- 12.4 日志
- 12.5 发布: 从本地包到 npm
- 12.6 发布前检查清单
- 12.7 一个「从零到运行」的收尾案例
- 12.8 本章小结

## 附录 A Cordis API 速查

- A.1 Context
- A.2 插件形态
- A.3 Service
- A.4 事件
- A.5 生命周期状态 (Fiber)
- A.6 错误
- A.7 作用域符号

## 附录 B DSH 服务目录 (ctx.\*)

- B.1 注册表与执行
- B.2 Agent 与会话
- B.3 安全与副作用
- B.4 模型与网络
- B.5 横切与基础设施
- B.6 消费方式对照

# DeepSeek Harness 插件开发指南

一本面向工程师的实战手册：从 Cordis 插件框架到 Profile、Bundle、工具、Skill、命令、系统提示词、Agent Preset 与客户端插件的完整开发路径，全部案例均来自 `@deepseek-ai/dsh` 真实代码库（版本 `0.1.0-rc.6`，核心框架 `@deepseek-ai/cordis@4.0.1`）。

## 这本书讲什么

DeepSeek Harness（简称 **DSH**）是一个基于 **Cordis** 插件框架构建的 AI Agent 运行时。它的每一块能力——模型路由、文件系统、沙箱、工具、Skill、命令、会话持久化、Web 界面——**都是一个可独立加载、可组合、可配置、可热更新的插件**。

这意味着：**学会写一个 DSH 插件，就等于学会了扩展整个 Agent 运行时的方法**。你可以给它加一个新工具（tool）、加一组可复用的技能（skill）、加一个斜杠命令（command）、注入一段系统提示词、组合一套专属的 Agent 预设（preset），甚至给浏览器界面加一个新的设置面板。

这本书把这一整套机制拆开讲透，并且**每一个案例都对应真实代码库里的实现**，而不是凭空捏造的伪代码。

## 读者与前置条件

- **读者**：想要扩展 DeepSeek Harness、或者想要理解它内部架构的 TypeScript/JavaScript 工程师。
- **前置条件**：熟悉 ES Module、TypeScript 类型、`async / await`、npm/pnpm 基础。了解依赖注入（DI）会让第 2、4 章读起来更快。
- **环境**：Node.js  $\geq 22$ ，pnpm。Cordis 是 ESM-first；脚手架要求 Node 22+。

## 阅读路径

1. **只想快速上手写一个工具**：读第 2 章（Cordis 核心）→ 第 3 章（bundle/patch）→ 第 5 章（工具开发）。这三章连起来就是一条最短可交付路径。
  2. **想全面掌握扩展机制**：按顺序读完全书。
  3. **正在调试某个插件**：直接查附录 A、附录 B，再回到对应章节看案例。
- 

## 关于「真实案例」

书中所有代码都严格遵循 `@deepseek-ai/dsh` 的真实约定，来源包括但不限于：

- `@deepseek-ai/cordis` 的 README 与核心源码（`Context` / `Service` / `Fiber` / 事件/生命周期）
- `@deepseek-ai/dsh-tool-todo`（工具插件的完整范例：  
`name` / `inject` / `Config` / `apply` / `defineTool`）
- `@deepseek-ai/dsh-command-goal`（命令插件的完整范例）
- `@deepseek-ai/dsh-skill-filessystem`（Skill 提供方范例）
- `@deepseek-ai/dsh-base` 的 `cordis.patch.yml`（bundle patch 层范例）
- `config/agent-presets/standard` 的 `preset.yml` 与 `agent.cordis.yml`（Agent Preset 范例）
- `@deepseek-ai/dsh-system-prompt`、`@deepseek-ai/dsh-tools`、`@deepseek-ai/dsh-client-modules` 等核心服务

凡是直接从真实代码精简而来的例子，我都会标注「**源自：**」并指出原始包名，方便你回源码对照。为可读性，代码做了适度缩略与注释改写，但**结构与 API 签名保持原样**。

---

本书由 *DeepSeek Harness* 代码库逆向梳理而成，面向教学；API 以你安装的实际版本为准（`dsh --version`）。

# 第 1 章 体系总览

---

这一章不写代码，先建立一张「DSH 到底是怎么拼起来的」的完整心智地图。理解了这张图，后面每一章的代码才有落点。

---

## 1.1 DeepSeek Harness 是什么

DeepSeek Harness (DSH) 是一个 **AI Agent 运行时**。它负责：

1. 组合出一棵「能力树」——哪些服务、哪些工具、哪些命令、哪些界面被加载；
2. 启动 Agent 循环 (agent loop)：接收用户输入 → 组装系统提示词 → 调用大模型 → 执行模型请求的工具 → 把结果写回历史 → 循环；
3. 管理会话 (session)、目标 (goal)、子代理 (subagent)、后台任务 (jobs)、文件沙箱、审批等横切能力；
4. 通过不同「入口模式」暴露：`dsh web` (浏览器)、`dsh --profile headless` (命令行一次性任务) 等。

DSH 与大多数单体 AI 应用最大的不同在于：**它没有任何硬编码的「功能列表」**。它的功能全部来自插件，而插件的「集合与顺序」由 **profile** 决定。

---

## 1.2 三层架构

DSH 自底向上分为三层：



|                                                                                                                                  |
|----------------------------------------------------------------------------------------------------------------------------------|
| <b>profile (组合层)</b><br>dsh.profile.bundles ← 有序的 bundle 列表<br>cordis.patch.yml ← 用户自己的 patch 层<br>--patch 覆盖层                   |
| <b>bundle (打包层)</b><br>一个 npm 包, package.json 声明 dsh.bundle.patch<br>内含 cordis.patch.yml, 声明一组「插件行」                              |
| <b>plugin (实现层)</b><br>一个 Cordis 插件: name / inject / Config / apply(ctx, cfg)<br>提供 Service / 注册 Tool / 注册 Command / 贡献 Prompt 段 |

对应到真实代码库:

- **plugin**: @deepseek-ai/dsh-tool-todo 就是一个插件, 导出 name、inject、Config、apply, 在 apply 里调用 ctx.tools.register(...)。
- **bundle**: @deepseek-ai/dsh-base 就是一个 bundle, 它的 package.json 里有 "dsh": { "bundle": { "patch": "./cordis.patch.yml" } }, 而那个 cordis.patch.yml 用 - insert: 列出几十个插件行 (timer、llm、session、tool-fs ..... )。
- **profile**: web 是一个 profile, 它的 dsh.profile.bundles 是 ["@deepseek-ai/dsh-base", "@deepseek-ai/dsh-web-app"]。

一句话概括: **plugin** 是「做什么」, **bundle** 是「打包哪一批 plugin」, **profile** 是「组合哪几批 bundle + 用户的覆盖」。

## 1.3 插件有几种「形态」

从**实现形态**看, Cordis 插件是一个函数、类或对象 (详见第 2 章)。从**功能角色**看, DSH 里的插件大致分这几类:

|    |     |     |    |                    |             |                            |                                  |       |             |                                     |                           |
|----|-----|-----|----|--------------------|-------------|----------------------------|----------------------------------|-------|-------------|-------------------------------------|---------------------------|
| 类别 | 干什么 | 典型包 | 章节 | ---  ---  ---  --- | <b>服务插件</b> | 用 ctx.provide() 暴露一个可注入的服务 | dsh-session、dsh-llm、dsh-fs-local | 第 4 章 | <b>工具插件</b> | 用 ctx.tools.register() 注册一个模型可调用的工具 | dsh-tool-todo、dsh-tool-fs |
|----|-----|-----|----|--------------------|-------------|----------------------------|----------------------------------|-------|-------------|-------------------------------------|---------------------------|

第 5 章 || **Skill 提供方** | 用 `ctx.skills.registerProvider()` 贡献技能来源 | `dsh-skill-filesystem` | 第 6 章 || **命令插件** | 用 `ctx.commands.register()` 注册斜杠命令 | `dsh-command-goal` | 第 7 章 || **提示词插件** | 用 `ctx.systemPrompt.section()` 贡献系统提示词段 | `dsh-plan-mode` | 第 8 章 || **组合插件 (bundle)** | 用 `cordis.patch.yml` 声明一批插件行 | `dsh-base`、`dsh-web-app` | 第 3 章 || **客户端插件** | 用 `dsh.client` manifest 在浏览器侧加载 | `dsh-client-ui-settings-plugins` | 第 10 章 |

注意：一个插件可以同时扮演多个角色。例如 `dsh-tool-todo` 既注册了工具，又（在有 `sessionProjections` 服务时）注册了一个会话投影单元。真正决定插件「是什么」的，是它 `apply()` 里调用了哪些 `ctx.*` 接口。

---

## 1.4 一次启动的完整数据流

以 `dsh web` 为例，理解「从敲命令到 Agent 能干活」的全链路：

```

dsh web
| 1. 启动器 (apps/cli) 只解析自己的 flag, 其余交给 web app
▼
composeProfile("web")
| 2. 找到 $DSH_HOME/profiles/web
| 3. 读取 dsh.profile.bundles = [dsh-base, dsh-web-app]
| 4. 按顺序取每个 bundle 的 cordis.patch.yml
| 5. 叠加 profile 自己的 cordis.patch.yml
| 6. 叠加 $DSH_HOME/cordis.patch.yml
| 7. 叠加 --patch 覆盖层
▼
boot(...) 通过 cordis-plugin-loader 逐行加载插件
| 8. 每个「行」{id, name, config, disabled} → 加载一个插件
| 9. Cordis 按 inject 依赖解析启动顺序 (激活是「服务可用性驱动」的)
| 10. 每个插件在 apply() 里注册服务/工具/命令/提示词段
▼
挂载的 app 插件 (web-server、api-proxy、agent-loop...)
| 11. 启动 HTTP 服务, 注入 window.__DSH_BOOT__ 客户端引导清单
▼
用户在浏览器发消息
| 12. agent loop 组装系统提示词 (systemPrompt.assemble)
| 13. 工具目录来自 ToolRuntime (所有 ctx.tools.register 的合集)
| 14. 调模型 → 模型返回工具调用 → ToolRuntime.execute 走执行流水线
| 15. 结果写回 session, 循环

```

其中 **第 3~7 步 (patch 层组合)** 和 **第 9~10 步 (Cordis 依赖解析)** 是理解「为什么一个插件装上就能用」的两把钥匙。前者回答「加载什么」, 后者回答「以什么顺序、什么时候可用」。

## 1.5 四个贯穿全书的核心理念

在进入代码之前, 先记住四个贯穿 DSH 与 Cordis 的设计原则, 它们会在后面反复出现:

### 1. 一切皆插件, 组合即配置

没有「内置功能」与「插件」之分。核心能力 (模型路由、文件系统、沙箱) 也只是一行 `cordis.patch.yml` 里的插件。你「裁剪 Harness」的方式不是改框架代码, 而是**组合不同的 bundle、patch 不同的行**。

## 2. 依赖注入 + 作用域 (scope)

插件通过 `inject` 声明自己需要哪些服务；Cordis 保证「被依赖的服务先启动、后销毁」。作用域（`isolate / extend`）让同一个服务名可以在不同隔离域内拥有不同实例——这是 Agent Preset 能「每个会话一份插件实例」的底层机制。

## 3. 生命周期即资源所有权

插件在 `apply()` 里创建的任何资源（定时器、监听器、服务、watcher），都要通过 `ctx.effect()` 登记。Cordis 保证插件被卸载（dispose）时，这些资源按**逆序**被干净地回收。这几乎是 DSH 插件「可热更新、可反复挂载卸载」的全部秘密。

## 4. 面向模型的表面是契约

工具的参数 schema、输出 schema、系统提示词文本、Skill 的 `name / description`——这些是**直接呈现给模型的文本与约束**。它们不仅影响模型能否正确调用，还影响 KV Cache 前缀稳定性（schema 变了，缓存从头失效）。所以 DSH 对「模型可见表面」有严格的确定性要求（见第 5、8 章的「模型体验」小节）。

# 1.6 关键术语表

| 术语 | 含义 | ---|---| **Cordis** | DSH 依赖的插件框架（@deepseek-ai/cordis），提供 Context、Service、依赖注入、生命周期。 || **Context (ctx)** | 一个依赖容器/作用域。插件通过它读写服务、注册能力、监听事件。 || **Service** | 用 `ctx.provide()` 暴露、用 `inject` 消费的命名对象（如 `ctx.fs`、`ctx.tools`）。 || **Fiber** | 一个插件实例的运行期记录：跟踪依赖状态、配置、生命周期 effects、清理。 || **plugin** | 插件本体：{ `name, inject, Config, apply` }（或函数/类形态）。 || **bundle** | 声明 `dsh.bundle.patch` 的 npm 包，用 `cordis.patch.yml` 打包一批插件行。 || **profile** | 一组有序 bundle + 用户 patch 层的组合，位于 `$DSH_HOME/profiles/<name>`。 || **patch** | 一段 YAML 列表，对插件行做 `insert / 覆盖`（id 定位，后者覆盖前者）。 || **tool** | 模型可调用的函数，含参数 schema、输出 schema、执行器。 || **skill** | 一段可被模型按需加载的指令/知识（SKILL.md）。 || **command** | 人类在输入框里敲的斜杠命令（如 `/goal`）。 || **preset** | 一组 Agent 面（agent-plane）的插件组合（`preset.yml + agent.cordis.yml`）。 |

下一章直接进入 Cordis 框架核心，写出**第一个真正可运行的插件**。

## 第 2 章 Cordis 框架核心

DSH 的一切插件都是 Cordis 插件。这一章把 Cordis 的核心概念讲透：Context、Service、插件形态、inject、provide、effect、事件、生命周期、配置校验。读完你能写出一个**真正可运行**的最小插件。

Cordis 是 DSH 从 cordis 项目（作者 Shigma）引入并维护在 vendor/cordis 的插件框架，发布为 @deepseek-ai/cordis。它是 ESM-first 的 TypeScript 框架。

### 2.1 Context：一切的起点

Context 是 Cordis 的**根依赖容器**。它同时是：

- 一个**服务解析器**：ctx.<serviceName> 读取已提供的服务；
- 一个**插件注册入口**：ctx.plugin(...)、ctx.inject(...)；
- 一个**作用域**：ctx.extend() / ctx.isolate() / ctx.intercept() 派生子作用域；
- 一个**事件总线**：ctx.on/emit/parallel/serial/bail/waterfall。

真实代码（@deepseek-ai/cordis 的 README Quick Start，也是全框架最简示例）：

```

import { Context, Service } from '@deepseek-ai/cordis'

// 1) 声明模块扩充：告诉类型系统 ctx 上多了哪些服务、事件
declare module '@deepseek-ai/cordis' {
  interface Context {
    counter: Counter
  }
  interface Events {
    'app/ready'(message: string): void
  }
}

// 2) 定义一个服务
class Counter extends Service {
  value = 0
  constructor(ctx: Context) {
    super(ctx, 'counter') // 立即注册为 ctx.counter，随 fiber 自动卸载
  }
  next() {
    return ++this.value
  }
}

// 3) 一个函数形态的插件，声明依赖 'counter'
const greeter = Object.assign((ctx: Context) => {
  ctx.on('app/ready', (message) => {
    ctx.logger.info('%s #%d', message, ctx.counter.next())
  })
}, {
  inject: ['counter'],
})

// 4) 装配
const root = new Context()
await root.plugin(Counter) // 先提供 counter 服务
await root.plugin(greeter) // 再启动依赖它的插件

root.emit('app/ready', 'started') // 打印 "started #1"
await root.fiber.dispose() // 逆序回收所有资源

```

这段代码展示了四个关键事实：

1. `new Context()` 创建根容器；

2. `ctx.plugin()` 启动插件并返回一个 `Fiber` ( `await` 它可等启动完成、抛错时得到启动错误) ;
3. `inject` 告诉 Cordis 该插件需要哪些服务, 服务先于消费者可用;
4. `effects`、事件监听器、服务, 都会在所属 fiber 被 `dispose` 时移除。

## 2.2 插件的三种写法

Cordis 的 `registry.resolve(plugin)` 接受三种形态, 最终都解析到一个「回调函数」作为插件的身份标识:

### 2.2.1 函数插件 (DSH 最常用的形态)

DSH 插件标准导出是**具名导出**, 由 loader 读取:

```
// packages/extensions/tool-todo/src/index.ts (节选, 真实代码)
import z from '@deepseek-ai/schemastery'
import { defineTool } from '@deepseek-ai/dsh-tools'

export const name = 'tool-todo'
export const inject = ['tools'] // 依赖 ctx.tools
export const Config = z.object({
  allowParallelInProgress: z.boolean().required(),
})

export function apply(ctx: Context, config: z.infer<typeof Config>) {
  // config 已通过标准 schema 校验
  ctx.tools.register(defineTool({ ... }))
}
```

要点:

- `name`: 插件名 (loader 用它做诊断/日志) 。
- `inject`: 字符串数组, 声明依赖的服务名。Cordis 会**等到这些服务可用**才执行 `apply` 。
- `Config`: Schemastery 的配置 schema, 启动前校验配置, 失败即 `ValidationError` 。
- `apply(ctx, config)`: 插件主体, `config` 是**已校验并归一化**后的配置对象。



## 2.2.2 类插件

类如果 `isConstructor` 成立（有 prototype、非生成器函数），会被 `new` 构造：

```
class MyPlugin {
  static inject = ['logger']
  constructor(ctx: Context, config: Config) {
    ctx.on('something', () => { /* ... */ })
  }
}
```

`Service` 就是类插件的典型基类：它在 `super(ctx, name)` 时立刻 `ctx.provide(name, this)`。

## 2.2.3 对象插件

```
const plugin = {
  name: 'my-plugin',
  inject: ['tools'],
  Config: z.object({ /* ... */ }),
  apply(ctx: Context, config) { /* ... */ },
}
```

`registry.resolve` 会取对象的 `.apply` 作为身份回调。

统一结论：无论哪种写法，`inject`（依赖）与 `Config`（配置 schema）都挂在插件身份（函数/类/对象）上，`apply`（或类构造）是执行体。

## 2.3 服务：provide / inject / Service

### 2.3.1 提供与消费

```
// 提供：把一个值注册为命名服务，随当前 fiber 自动注销
const dispose = ctx.provide('myService', someValue)

// 消费：声明依赖后，在 apply 里直接读
export const inject = ['myService']
export function apply(ctx) {
  ctx.myService.doSomething()
}
```

Cordis 的服务解析是**惰性 + 依赖驱动**的：

- 一个插件 `inject` 了某服务，但该服务尚未提供 → 插件保持 `inactive`（未激活）状态，**不执行** `apply`。
- 服务被提供 → Cordis `notify` 所有依赖它的 fiber，重新评估，满足即激活。
- 服务被注销 → 依赖它的插件先卸载，等新服务实例出现再重启。

这就是 DSH `cordis.patch.yml` 注释里反复强调的「**激活是服务可用性驱动，行顺序不承载加载语义**」。

### 2.3.2 Service 基类

```
class MyService extends Service {
  constructor(ctx: Context) {
    super(ctx, 'myService') // 等价于 ctx.provide('myService', this)
  }
}
```

`Service` 的构造器里调用 `ctx.reflect.provide(name, this, this[Service.check])`，因此：

- 实例随所属 fiber 自动注册与注销；
- 可定义 `static provide = 'xxx'` 或 `static check`（可用性谓词）。

### 2.3.3 惰性读取： `ctx.get`

```
const fs = ctx.get('fs') // 返回 undefined，若未提供；不抛错
```

DSH 里很多可选依赖都用 `ctx.get('approval')` 这种「**无静态注入**」方式消费：有没有审批服务都能跑，有就走审批，没有就退化（fail-closed）。这是它与 `inject`（硬依

赖，没有就不启动) 的关键区别。

## 2.4 effect：资源所有权与清理

Cordis 的**生命周期铁律**：插件在 `apply` 里创建的任何需要回收的资源，都要交给 `ctx.effect()` 登记。插件被卸载时，effects 按**逆序**执行清理。

`ctx.effect(execute, label)` 的 `execute` 可以是：

1. **同步函数**，返回一个 disposer：

```
ctx.effect(() => {
  const timer = setInterval(tick, 1000)
  return () => clearInterval(timer) // 卸载时调用
}, 'my-timer')
```

2. **async 函数**，返回一个 async disposer：

```
ctx.effect(async () => {
  const client = await connect()
  return async () => await client.close()
})
```

3. **生成器函数**，用 `yield` 依次登记多个 disposer（真实代码常用形态）：

```
// 源自 dsh-skill-filessystem/src/index.ts
ctx.effect(function* () {
  yield async () => { await provider.dispose() }
}, 'skill-filessystem watcher')
```

生成器里每个 `yield` 的值都是一个 disposer；卸载时逆序调用。

4. **直接返回一个 disposer 函数。**

DSH 全库几乎把「资源生命周期」全部建立在 effect 上：定时器、事件监听、watcher、服务注册、子进程.....**只要你把清理闭包交给 effect，框架就替你保证不会泄漏。**

## 2.5 事件总线

Cordis 内置了六种事件分发方式，覆盖了 DSH 里绝大多数「扩展点」：

| 方法                                    | 语义                                        | 典型用途      |
|---------------------------------------|-------------------------------------------|-----------|
| <code>ctx.on(name, listener)</code>   | 注册监听器（随 fiber 自动注销）                       | 订阅生命周期/通知 |
| <code>ctx.once(name, listener)</code> | 只触发一次                                     | 一次性初始化    |
| <code>ctx.emit(...args)</code>        | 同步并发分发，不等待 Promise                        | 通知        |
| <code>ctx.parallel(...args)</code>    | 并发分发并等待全部，收集 rejection                    | 广播 + 汇总错误 |
| <code>ctx.serial(...args)</code>      | 顺序 await，直到某个返回值「bail」                    | 链式查找      |
| <code>ctx.bail(...args)</code>        | 同步顺序，直到某个返回值「bail」                        | 同步短路      |
| <code>ctx.waterfall(...args)</code>   | 顺序执行，最后一个参数是 <code>next</code> ；监听器可拦截/改写 | 变换/拦截扩展点  |

「bail」的定义：返回值不是 `null / false / undefined` 即视为「命中并停止」。

**waterfall 是 DSH 扩展点的主力机制。** 比如 `tools/execute`、`tools/pre-execute`、`tools/post-execute`、`system-prompt/assemble`、`internal/config`、`internal/update` 都是 waterfall：中间件按顺序包裹，可改写参数、可提前 veto、可替换结果。

## 2.6 生命周期：Fiber 的状态机

每个插件实例都对应一个 `Fiber`。它的状态（`state` 字段）：

| state | 常量                     | 含义                               |
|-------|------------------------|----------------------------------|
| 0     | <code>inactive</code>  | 未激活（依赖未满足，或已卸载）                  |
| 1     | <code>loading</code>   | 正在加载（ <code>_reload</code> 执行中）  |
| 2     | <code>active</code>    | 活跃（ <code>apply</code> 已完成）      |
| 3     | <code>error</code>     | 启动失败（ <code>_error</code> 记录了原因） |
| 4     | <code>disposed</code>  | 已 dispose                        |
| 5     | <code>unloading</code> | 正在卸载                             |

关键方法：

- `fiber.await()`：等待当前生命周期工作结束；若有启动错误则抛出。
- `fiber.dispose()`：卸载插件（逆序清理 effects）。
- `fiber.restart()`：用当前配置卸载并重载。
- `fiber.update(config)`：校验新配置，走 `internal/update` waterfall（HMR 可 veto），然后 restart。

- `fiber.getEffects()`：诊断用，返回当前 effect 树。

理解这些状态对调试至关重要：你看到的「插件没生效」，多半是它停在 `inactive`（依赖没满足）或 `error`（启动抛错）状态。第 12 章会讲如何诊断。

## 2.7 配置校验：Schemastery + 标准 Schema

DSH 用 `@deepseek-ai/schemastery` 定义配置 schema。`Config` 通过 **standard-schema** 接口暴露 `["~standard"].validate`，Cordis 在启动前调用它，失败抛 `ValidationError`（聚合所有 issue）。

```
import z from '@deepseek-ai/schemastery'

export const Config = z.object({
  providerName: z.string().min(1).default('filesystem'),
  includeDefaultRoots: z.boolean().default(true),
  customSkillDirs: z.array(z.string()).default([]),
  watch: z.boolean().default(true),
})
```

要点：

- `.default(x)` 意味着该字段可缺省，归一化后 `config` 上一定有个值。
- `.required()` 意味着缺省即校验失败。
- 校验是**同步的**（`"then" in result` 会抛 `TypeError("Async config validation is not supported")`）。

因此插件作者可以假设 `apply` 拿到的 `config` 一定是良构的，不必再手工判空。

## 2.8 作用域：extend / isolate / intercept

Context 可以派生子作用域，且**不修改父作用域**：

```
ctx.extend(meta)           // 子上下文, meta 的 own 属性覆盖继承属性
ctx.isolate(name, label)    // 为某个服务名开辟独立服务作用域 (同 label 会 join)
ctx.intercept(name, config) // 为某个服务附加 intercept 配置 (Service.resolveConfig
```

作用域的威力在第 9 章 (Agent Preset) 体现: `agent.cordis.yml` 里的 `isolate: { planMode: true }` 就是为 `planMode` 服务开辟**每预设一份实例**的隔离域, 避免多个预设共享同一实例。这里先记住概念即可。

## 2.9 第一个可运行插件：把它们串起来

下面是一个**完整可运行**的最小插件, 它演示了 `name / inject / Config / apply / effect / 事件 / 服务全链路`。它做的事: 提供一个 `clock` 服务, 每秒 `emit` 一个 `tick` 事件; 另一个插件消费该事件并打印。

```
// clock-plugin.ts
import { Context, Service } from '@deepseek-ai/cordis'
import z from '@deepseek-ai/schemastory'

declare module '@deepseek-ai/cordis' {
  interface Context { clock: Clock }
  interface Events { 'clock/tick'(at: string): void }
}

class Clock extends Service {
  constructor(ctx: Context) { super(ctx, 'clock') }
  now() { return new Date().toISOString() }
}

export const name = 'clock-provider'
export const Config = z.object({ intervalMs: z.number().default(1000) })

export function apply(ctx: Context, config: z.infer<typeof Config>) {
  ctx.plugin(Clock) // 提供服务
  ctx.effect(() => {
    const timer = setInterval(() => ctx.emit('clock/tick', ctx.clock.now()), config.intervalMs)
    return () => clearInterval(timer) // 卸载时清定时器
  }, 'clock ticker')
}
```

```
// tick-consumer.ts
export const name = 'tick-consumer'
export const inject = ['clock'] // 等 clock 服务就绪才启动

export function apply(ctx: Context) {
  ctx.on('clock/tick', (at) => ctx.logger.info('tick at %s', at))
}
```

```
// main.ts
import { Context } from '@deepseek-ai/cordis'
import * as clockProvider from './clock-plugin.js'
import * as tickConsumer from './tick-consumer.js'

const root = new Context()
await root.plugin(clockProvider)
await root.plugin(tickConsumer)

// 进程会持续每秒打印 tick; 按 Ctrl+C 退出。
```

在 DSH 里，你不会手动 `root.plugin(...)`，而是把插件名写进 `cordis.patch.yml`（下一章）。但这个手动装配模型，正是 DSH 加载器的底层。

## 2.10 本章小结

| 概念 | 一句话 | |---|---| | Context | 依赖容器 + 服务解析器 + 作用域 + 事件总线 | | 插件形态 | 函数 / 类 / {apply}，都带 inject + Config | | Service | super(ctx, name) 即注册，随 fiber 注销 | | inject | 硬依赖：没有就不启动；ctx.get 是软依赖 | | effect | 资源所有权：登记清理闭包，逆序回收 | | 事件 | on/once/emit/parallel/serial/bail/waterfall 六种分发 | | Fiber | 插件运行期：状态机 + 启动/卸载/重载 | | Config | Schemastery 同步校验，apply 拿到良构配置 |

下一章讲 profile、bundle、patch 层——即「如何把这棵插件树**组合并配置**起来」。



## 第 3 章 Profile、Bundle 与 patch 层

这一章回答一个核心问题：**你写好的插件，如何进入 DSH 并被加载？** 答案是三件套：  
bundle（打包插件行）、profile（组合 bundle）、patch 层（覆盖与配置）。

### 3.1 目录结构与三份文件

一个 profile 是一个目录，位于 `$DSH_HOME/profiles/<name>`（`$DSH_HOME` 默认 `~/.dsh`，可通过环境变量覆盖）。首次使用 `web / headless` 或执行 `dsh plugin` 时会自动初始化，得到三份文件：

```
$DSH_HOME/profiles/<name>/
├─ package.json          # 依赖 + profile manifest (dsh.profile.bundles)
├─ cordis.patch.yml      # 用户自己的 patch 层
├─ pnpm-workspace.yaml   # pnpm 配置 (nodeLinker: hoisted 等)
└─ node_modules/        # 树外插件由 pnpm 安装到这里
```

真实代码（`@deepseek-ai/dsh-app-boot` 的 `initProfile`）生成的结构：

```
// package.json（真实模板）
{
  "name": "dsh-profile-<name>",
  "private": true,
  "dependencies": {},
  "dsh": { "profile": { "bundles": ["@deepseek-ai/dsh-base"] } }
}
```

```
# cordis.patch.yml（真实模板）
# 你的 patch 层，应用在所有 bundle 层之后：一个顶层 YAML 数组，
# 含 loader patch 条目（按 id 定位的配置覆盖、disable、insert 列表；允许 !!js 表达式
[]
```

```
# pnpm-workspace.yaml (真实模板)
packages:
  - .

nodeLinker: hoisted
autoInstallPeers: false
```

## 3.2 三层 patch 的叠加顺序

DSH 的配置树从「空根」开始，按以下顺序叠加（**后者覆盖前者**）：

1. `dsh.profile.bundles` 中**每个 bundle 的 patch**（按 bundles 列表顺序）；
2. profile 自身的 `cordis.patch.yaml`；
3. home 级的 `$DSH_HOME/cordis.patch.yaml`（机器级偏好，作用于所有 profile，所以它高于 per-profile 层）；
4. `--patch` 指定的覆盖层（argv 顺序）；
5. 启动器派生的 flag patch（如遥测禁用开关）。

真实代码（`@deepseek-ai/dsh/lib/profile-boot-*.js` 的 `composeProfile`）明确了这个顺序。**层优先级越高，越靠后应用，越有最终决定权。**

理解这个顺序的实用价值：你想「覆盖一个 bundle 里某插件的配置」，就写在 profile 的 `cordis.patch.yaml`；你想「本机所有 profile 都关掉某个东西」，就写在 `$DSH_HOME/cordis.patch.yaml`；你想「临时试一个覆盖」，就用 `--patch`。

## 3.3 patch 条目的语义

patch 文件是一个**顶层 YAML 数组**，元素是 loader patch 条目。最常用的是 `insert` 与「按 id 覆盖」。

### 3.3.1 insert：声明插件行

这是 bundle 的核心内容。真实代码（@deepseek-ai/dsh-base/cordis.patch.yml 节选）：

```
- insert:
  - id: timer
    name: '@deepseek-ai/cordis-plugin-timer'

  - id: llm
    name: '@deepseek-ai/dsh-llm'

  - id: agent-default-model
    name: '@deepseek-ai/dsh-agent-default-model'
    config:
      provider: deepseek-official
      model: deepseek-v4-flash

  - id: bash-sandbox
    name: '@deepseek-ai/dsh-bash-sandbox'
    disabled: !!js process.platform === 'win32'
    config:
      timeoutMs: 60000
```

每一行的字段：

| 字段       | 含义                                    |
|----------|---------------------------------------|
| id       | 该行的 <b>寻址标识</b> ，后续层按它覆盖/禁用           |
| name     | 插件包名（loader 据此解析模块）                   |
| config   | 传给插件 apply 的配置对象（会被 Config schema 校验） |
| disabled | 为真则该行不加载（可用 !!js 表达式按平台/环境动态决定）       |

注意两点：

- config 是整体替换，不是深合并。**后一个层写了同一 id 的 config，就完全替换前一个层的 config。所以「按模式不同而不同的行」不属于共享 bundle（base），而是每个模式 bundle 各自完整声明（这是 dsh-base 头部注释明确说明的约定）。
- 行顺序不承载加载语义。**激活是「服务可用性驱动」的（第 2 章），行顺序只是给读者分组。

### 3.3.2 按 id 覆盖与禁用

用户在自己的 cordis.patch.yml 里，可以引用 bundle 已声明的 id 来覆盖：

```
# profile 的 cordis.patch.yml — 覆盖 base 里 session-query-sqlite 的配置
- id: session-query-sqlite
  config:
    path: '/data/sessions.db'
    openAt: first-search

# 禁用某行
- id: skill-badge
  disabled: true
```

`!!js` 表达式让配置可以依赖运行环境。真实代码里大量使用：

```
- id: bash-sandbox
  name: '@deepseek-ai/dsh-bash-sandbox'
  disabled: !!js process.platform === 'win32'      # Windows 上禁用 bash

- id: pwsh-sandbox
  name: '@deepseek-ai/dsh-pwsh-sandbox'
  disabled: !!js process.platform !== 'win32'     # 非 Windows 上禁用 pwsh
```

### 3.4 bundle: dsh.bundle.patch 声明

一个 bundle 就是一个 npm 包，它的 `package.json` 里声明了 patch 文件的路径：

```
// @deepseek-ai/dsh-base/package.json (真实节选)
{
  "name": "@deepseek-ai/dsh-base",
  "dsh": { "bundle": { "patch": "./cordis.patch.yml" } },
  "exports": {
    ".": { "types": "./lib/types/index.d.ts", "default": "./lib/index.js" },
    "./cordis.patch.yml": "./cordis.patch.yml"
  }
}
```

`dsh plugin` 的**自动对账** (reconcile) 机制就依赖这个声明：一个依赖包如果解析到了 `dsh.bundle.patch`，它就会自动加入 `dsh.profile.bundles`；如果某个已列出的依赖不再声明（被卸载，或新版本删掉了声明），就从列表移除。真实代码（`@deepseek-ai/dsh/lib/plugin-*.js`）：

```
function exportsPatch(packageName, profileDir) {
  const dir = resolveBundleDir(NAME, packageName, INSTALL_ANCHOR, profileDir)
  return readProfileManifest(NAME, dir).dsh?.bundle?.patch !== void 0
}
```

## 3.5 模块解析：两锚点

`dsh.profile.bundles` 里的包名按以下顺序解析（真实代码注释明确）：

1. **先从** dsh 安装目录（launcher 自身的包）解析——`@deepseek-ai/dsh-base`、`@deepseek-ai/dsh-web-app`、`@deepseek-ai/dsh-headless` 等 in-box bundle 永远从这里来；
2. **再从** profile 自己的 `node_modules` 解析——pnpm 把树外插件装在这里。

这是「**bundles 来自安装**」契约的体现：内置 bundle 不经 pnpm 管理，而是通过 `$DSH_HOME/profiles/node_modules` 这个「每个安装内包一个符号链接」的扁平 fallback 目录，让任何 profile 都能用普通 Node 父目录向上查找解析到内置插件。

**实用含义：**你开发自己的 bundle，就 `dsh plugin --profile <name> add <你的包>`，让它进 profile 的 `node_modules`；它声明了 `dsh.bundle.patch`，就会被对账进 `bundles` 列表。

## 3.6 实战：创建并挂载自己的 bundle

目标：做一个最小 bundle `my-bundle`，里面只有一个插件（第 2 章的 clock/tick 可以复用，这里用一个更简单的「启动问候」插件），然后装进一个自定义 profile 并验证它被加载。

### 第 1 步：创建 bundle 包

```

my-bundle/
├─ package.json
├─ cordis.patch.yml
└─ lib/
    ├─ index.js      # 插件本体 (name/inject/apply)
    └─ index.d.ts    # 类型 (可选)

```

```

// package.json
{
  "name": "@you/my-bundle",
  "version": "0.1.0",
  "type": "module",
  "main": "lib/index.js",
  "exports": {
    ".": "./lib/index.js",
    "./cordis.patch.yml": "./cordis.patch.yml"
  },
  "dsh": { "bundle": { "patch": "./cordis.patch.yml" } },
  "peerDependencies": {
    "@deepseek-ai/cordis": "^4.0.1",
    "@deepseek-ai/schemastery": "*"
  }
}

```

```

# cordis.patch.yml — 声明一行插件
- insert:
  - id: my-greeter
    name: '@you/my-bundle'
    config:
      message: 'hello from my bundle'

```

```
// lib/index.js — 插件本体
import z from '@deepseek-ai/schemastery'

export const name = 'my-greeter'
export const Config = z.object({ message: z.string().default('hello') })

export function apply(ctx, config) {
  ctx.on('app/ready', () => {
    ctx.logger.info('[my-greeter] %s', config.message)
  })
}
```

## 第 2 步：初始化 profile 并安装 bundle

```
# 自定义 profile（非 web/headless 模板，用 dsh plugin 触发初始化）
dsh plugin --profile mypro add @you/my-bundle
```

这条命令会：

1. 若 `$DSH_HOME/profiles/mypro` 不存在，用 `DEFAULT_PROFILE_BUNDLES = ["@deepseek-ai/dsh-base"]` 初始化；
2. 把 cwd 切到 profile 目录，执行 `pnpm add @you/my-bundle`；
3. 执行 `reconcilePlugins`：发现 `@you/my-bundle` 声明了 `dsh.bundle.patch`，自动把它 append 到 `dsh.profile.bundles`。

之后 `package.json` 会变成：

```
{
  "dependencies": { "@you/my-bundle": "^0.1.0" },
  "dsh": { "profile": { "bundles": ["@deepseek-ai/dsh-base", "@you/my-bundle"] } }
}
```

## 第 3 步：验证配置树

```
dsh --profile mypro --dump-config
```

输出的是组合后的配置树——你应该能看到 `my-greeter` 这一行及其 config。 `--dump-default-config` 则只打印 bundle 层（不含用户层与 `--patch`）。

## 第 4 步：覆盖它的配置

在 `$DSH_HOME/profiles/mypro/cordis.patch.yml` 写：

```
- id: my-greeter
  config:
    message: 'overridden by my profile'
```

再 `dsh --profile mypro --dump-config` , `my-greeter` 的 `config.message` 就变成了覆盖值。

## 3.7 常见陷阱

1. **config 是整体替换**：想改某个 bundle 行里 config 的一个字段，必须完整重写该 config（参考该行的 schema 补全所有必填项），而不是只写要改的那个字段。
2. **bundle 名写在 bundles 里但包没声明 dsh.bundle.patch**：启动会报错 `declares no dsh.bundle`。别把普通库当成 bundle 写进列表。
3. **git 托管的插件需要 prepare 脚本构建**：`dsh plugin add git+https://...` 时，pnpm 默认会阻止依赖的构建脚本，报错时按提示把 pnpm 打印的 key 加到 profile 的 `pnpm-workspace.yaml` 的 `allowBuilds`（第 12 章详述）。
4. **相对路径 spec 会被锚定到调用目录**：`dsh plugin add .` 或 `../plugin` 会被解析为「相对于你敲命令时的 cwd」，避免误把 profile 自己 link 进来。

## 3.8 本章小结

- **bundle** = 声明 `dsh.bundle.patch` 的 npm 包，其 `cordis.patch.yml` 用 `- insert:` 声明插件行。
- **profile** = `dsh.profile.bundles`（有序 bundle）+ 自己的 `cordis.patch.yml`。
- **patch 层叠加顺序**：bundles → profile 层 → home 层 → `--patch`。
- **行寻址**：`{id, name, config, disabled}`；config 整体替换；`!!js` 支持表达式；行顺序无加载语义。
- **dsh plugin** = pnpm 转发 + 按 `dsh.bundle.patch` 自动对账 bundles。
- **模块解析两锚点**：安装目录 → profile `node_modules`。



下一章深入「服务与依赖注入」——插件之间如何协作、作用域如何隔离。

## 第 4 章 服务与依赖注入

DSH 的插件不是孤立运行的：它们通过**服务**互相协作。这一章把服务的提供、消费、作用域、以及「服务与插件生命周期的耦合」讲清楚，并给出一个完整的服务插件案例。

### 4.1 服务是 DSH 的「毛细血管」

在 DSH 里，几乎每个横切能力都是一个 `ctx.*` 服务：

| 服务名 | 提供方 | 用途 | |---|---|---| | `ctx.fs` | `dsh-fs-local` / `dsh-fs-sandbox` | 文件系统读写（走沙箱） || `ctx.tools` | `dsh-tools` | 工具注册表与执行流水线 || `ctx.skills` | `dsh-skill` | Skill 注册表 || `ctx.commands` | `dsh-commands` | 斜杠命令注册表 || `ctx.systemPrompt` | `dsh-system-prompt` | 系统提示词组装 || `ctx.session` | `dsh-session` | 会话（持久化日志） || `ctx.llm` | `dsh-llm` | 模型请求 || `ctx.approval` | `dsh-user-approval` | 审批（ask/allow/deny） || `ctx.credentials` | `dsh-credentials-local` | 凭据解析 || `ctx.goals` | `dsh-goal` | 持久化目标 |

**插件之间的协作几乎都通过服务完成：**工具插件 `inject: ['tools']` 然后 `ctx.tools.register(...)`；命令插件 `inject: ['commands', 'goals']` 然后 `ctx.commands.register(...)`。

### 4.2 提供服务： `ctx.provide` 与 `Service`

#### 4.2.1 直接用 `ctx.provide`

```
export function apply(ctx) {
  const myService = { hello: () => 'world' }
  ctx.provide('myService', myService) // 随本 fiber 自动注销
}
```

`ctx.provide` 返回一个 disposer，但通常不用手动调用——它已经登记为当前 fiber 的 effect，插件卸载时自动注销。

关键约束（真实代码 `ReflectService.provide`）：

- 同名服务在同一作用域内重复提供会报错：`service "x" has been registered at <fiber name>`。
- 提供服务的 fiber 必须是当前活跃 fiber；在已 dispose 的 fiber 上提供会抛 `INACTIVE_EFFECT`。
- 可传第三个参数 `check`：一个可用性谓词，依赖方会据此判断该服务是否「有效」。

#### 4.2.2 用 `Service` 基类

```
import { Service } from '@deepseek-ai/cordis'

class FilesystemService extends Service {
  static provide = 'fs' // 可选：默认服务名
  constructor(ctx) {
    super(ctx, 'fs') // 立即 ctx.provide('fs', this)
  }
  async readText(target) { /* ... */ }
}

export const name = 'my-fs'
export function apply(ctx) {
  ctx.plugin(FilesystemService)
}
```

`Service` 构造器里 `ctx.reflect.provide(name, this, this[Service.check])`，所以服务实例的生命周期 = 所属 fiber 的生命周期。

### 4.3 消费服务：`inject`（硬）与 `ctx.get`（软）

#### 4.3.1 `inject`：硬依赖

```
export const inject = ['fs', 'tools']
export function apply(ctx, config) {
  // 到这里，ctx.fs 和 ctx.tools 一定已可用
}
```

语义：

- Cordis 会**等到**所有 inject 的服务都可用才执行 `apply`。
- 服务注销时，依赖它的插件先卸载；新实例出现后自动重启。
- 这是「激活是服务可用性驱动」的直接体现。

### 4.3.2 `ctx.get`：软依赖

```
export function apply(ctx) {
  const approval = ctx.get('approval') // 可能 undefined
  if (approval) { /* 走审批 */ } else { /* 降级为拒绝 */ }
}
```

DSH 的官方约定是：**可选的、可能不部署的服务用 `ctx.get`，不用静态 `inject`**。真实例子是 `dsh-tools` 对审批 seam 的消费——「未部署该 seam 时仍会将询问退化为拒绝，而无论是否存在该 seam，注册表都保持活动」。

## 4.4 作用域与隔离：`isolate` / `intercept` / `extend`

### 4.4.1 为什么需要隔离

一个服务名默认是**进程级**的：谁提供、全进程共享一个实例。但 Agent Preset 需要「**每个预设（甚至每个会话）一份实例**」——否则一个预设的状态会污染另一个。

Cordis 用 `isolate` 解决：为某个服务名开辟一个**隔离作用域**，标签相同的 `isolate` 调用会 join 到同一作用域。

```
// 为 planMode 服务开辟「本作用域私有」的实例
const scoped = ctx.isolate('planMode', true)
```

`ReflectService.provide` 里：`this.ctx[symbols.isolate][name]` 决定服务实例存到哪个 key；不同 isolate 标签 → 不同 key → 互不覆盖。

### 4.4.2 `intercept`：服务级配置合并

`intercept` 给服务附加配置，供 `Service.resolveConfig()` 合并。典型场景是 `logger`：不同作用域可以给 `logger` 附加不同的 `name/level` 配置，`ctx.logger()` 会沿原型链收集所有 `intercept` 配置合并。

深入作用域机制看第 9 章（Agent Preset 的 `isolate realm`）。这里先建立概念：**同名服务可以因作用域不同而拥有不同实例。**

## 4.5 实战：一个计数器服务 + 消费者（可运行）

这是把第 2 章的 Counter 扩成「带配置、带作用域隔离、可观测」的完整案例，演示服务插件的标准骨架。

### 4.5.1 服务插件

```
// counter-service.ts
import { Context, Service } from '@deepseek-ai/cordis'
import z from '@deepseek-ai/schemastory'

declare module '@deepseek-ai/cordis' {
  interface Context { counter: Counter }
  interface Events { 'counter/changed'(value: number): void }
}

export class Counter extends Service {
  private value = 0
  constructor(ctx: Context, private readonly step = 1) {
    super(ctx, 'counter')
  }
  next() {
    this.value += this.step
    this.ctx.emit('counter/changed', this.value)
    return this.value
  }
  get() { return this.value }
}

export const name = 'counter-service'
export const Config = z.object({ step: z.number().default(1) })

export function apply(ctx: Context, config: z.infer<typeof Config>) {
  ctx.inject([], (c) => {
    // 通过一个临时的子作用域，让每个实例的 step 不同也能并存（演示 isolate）
    const scoped = c.isolate('counter')
    scoped.plugin(class extends Counter {
      constructor(ctx: Context) { super(ctx, config.step) }
    })
  })
}
```

说明：真实 DSH 里一般直接 `ctx.plugin(SomeService)`，这里的 `isolate` 只是为了演示「同名服务可在不同作用域并存」。多数服务插件不会自己 `isolate`。

#### 4.5.2 消费者插件

```
// counter-reporter.ts
export const name = 'counter-reporter'
export const inject = ['counter']

export function apply(ctx: Context) {
  ctx.on('counter/changed', (value) => {
    ctx.logger.info('counter now %d', value)
  })
}
```

### 4.5.3 装配与验证

```
import { Context } from '@deepseek-ai/cordis'
import * as counterService from './counter-service.js'
import * as counterReporter from './counter-reporter.js'

const root = new Context()
await root.plugin(counterService, { step: 2 })
await root.plugin(counterReporter)

ctx.counter.next() // 打印 "counter now 2"
ctx.counter.next() // 打印 "counter now 4"
await root.fiber.dispose()
```

在 DSH 里, 这一步由 `cordis.patch.yml` 的 `- insert:` 完成:

```
- insert:
  - id: counter-service
    name: '@you/counter-service'
    config: { step: 2 }
  - id: counter-reporter
    name: '@you/counter-reporter'
```

## 4.6 服务的生命周期耦合（重点理解）

Cordis 的服务与插件生命周期是**双向绑定**的:

提供方 fiber 卸载

- 服务从 store 移除
- notify 所有 inject 该服务的 fiber
- 这些 fiber 卸载（等待其 effects 清理）
- 若出现新提供方，这些 fiber 重启

这意味着：

1. **服务提供方崩了，依赖方会跟着卸载**，不会留下「悬空依赖」。
2. **热更新一个服务插件**，会级联重启所有依赖它的插件——这是 HMR 能安全工作的基础（第 12 章）。
3. **提供服务的插件应该在 `apply` 里一次性完成注册**，不要在 `apply` 之外异步 `provide`，否则时序不可控。

## 4.7 常见陷阱

1. **重复提供**：同一作用域内同名服务二次提供会抛错。多实例需求请用 `isolate` 开辟作用域，而不是硬塞同名。
2. **在已 `dispose` 的 fiber 上 `provide/effect`**：抛 `INACTIVE_EFFECT`。这通常发生在「异步回调在插件已卸载后仍然执行」。用 `ctx.effect` 登记清理，确保回调在卸载前注销。
3. **把「可选依赖」写成 `inject`**：会让插件在服务缺失时永远不激活。可选的用 `ctx.get`。
4. **服务状态跨会话泄漏**：进程级服务里若缓存了「每会话」状态，多个会话会互相污染。DSH 用「按 session/agent 键控」解决（`dsh-goal`、`dsh-jobs-local` 的 registry 都按 agent/session 键控）。你的服务若保存每会话状态，务必键控。

## 4.8 本章小结

- 提供：`ctx.provide(name, value)` 或 `Service` 基类；随 fiber 自动注销。
- 消费：`inject`（硬依赖，缺了不启动）与 `ctx.get`（软依赖，缺了降级）。
- 作用域：`isolate` 为服务名开辟独立实例域；`intercept` 做服务级配置合并。
- 生命周期：服务提供方卸载 → 依赖方级联卸载/重启。



- 状态键控：跨会话状态务必按 session/agent 键控。

下一章是全书重头戏——**工具开发**，DSH 里最常见的插件类型。

## 第 5 章 工具开发

工具 (tool) 是模型唯一能「动手」的通道。这一章是全书最实用的一章：讲透 `defineTool`、输出契约、执行上下文、执行流水线、UI 呈现，并给出多个可直接上线的真实案例。

### 5.1 工具注册表： `ToolRuntime` ( `ctx.tools` )

所有工具的入口是 `ctx.tools` 服务（提供方 `@deepseek-ai/dsh-tools`）。工具插件 `inject: ['tools']`，然后在 `apply` 里注册：

```
ctx.tools.register(definition) // 返回 disposer，随 fiber 自动注销
```

核心 API（真实契约）：

- `ctx.tools.register(definition: ToolDefinition): () => void` —— 注册一个受信任、带类型的工具，**必须**包含规范的 `output` 声明。注册随调用方 fiber 注销。
- `ctx.tools.get(name, scope?)` —— 解析指定作用域可见的工具定义（已应用名称遮蔽）。
- `ctx.tools.schemas(scope?)` —— 返回该作用域可见的所有 schema（不含 `execute`）。
- `ctx.tools.guard(guard)` —— 注册一个单调守卫（拒绝即最终拒绝）。
- `ctx.tools.presentAs(mode)` —— 为本 agent 选择呈现模式（`native/code/both`）。
- `ctx.tools.execute(exec)` —— 快照参数、走完整流水线执行。

**工具的「层」由调用上下文作用域决定：**普通插件上下文注册的是**全局**工具；

`agent.ctx` 注册的只为该 agent 可见，并在此遮蔽同名全局工具。同一层内重名抛错。

### 5.2 `defineTool`：类型化工具定义

DSH 为第一方插件提供 `defineTool()`，它做三件事：

1. 把统一的参数 schema DSL 编译成 JSON Schema;
2. 执行前校验模型参数 (缺失必填、类型错误、非法枚举 → `ToolArgsError` , 进入普通错误结果路径) ;
3. 依据 `output.schema` 推断执行器返回类型, 并在呈现前快照校验返回的**无损 JSON**。

### 5.2.1 最小示例 (官方 README, 真实代码)

```
import { readFile } from 'node:fs/promises'
import type { Context } from '@deepseek-ai/cordis'
import { defineTool } from '@deepseek-ai/dsh-tools'

declare const ctx: Context

ctx.tools.register(defineTool({
  name: 'read_file',
  description: 'Read a file from disk.',
  parameters: {
    path: { type: 'string', required: true, description: 'Absolute file path' },
    offset: { type: 'number' },
    limit: { type: 'number' },
  },
  output: {
    schema: { type: 'string' },
    render: (_args, value) => [{ type: 'text', text: value }],
  },
  async execute(args, exec) {
    // args 已被类型化: { path: string; offset?: number; limit?: number }
    return readFile(args.path, { encoding: 'utf8', signal: exec.signal })
  },
}))
```

### 5.2.2 参数 schema DSL

`parameters` 是一个「隐式开放参数对象」, 每个属性支持:

| type | 说明 | |---|---| | `string` / `number` / `integer` / `boolean` / `null` | 标量 (可加 `enum` / `const`) | | `array` | 数组 ( `items` 指定元素 schema) | | `object` | 对象 (**必须显式声明** `additionalProperties: true/false` , `properties` 定义字段) | | `json` | 任意 JSON 值 (仅作者可用) | | `oneOf` | 恰好匹配一个分支 |

每个属性字段可带 `required: true` 和 `description` 。关键规则：

- **隐式参数根是开放的**（多余键默认接受）；
- 显式对象只有在 `additionalProperties: true` 时才接受额外键；封闭对象（`false`）只接受声明的属性；
- 不应用默认值（模型必须显式传参）。

### 5.2.3 输出契约（最重要）

每个工具必须声明 `output.schema`（一个 JSON Schema，注册表强制校验执行器返回的规范值）：

```
output: {
  schema: { type: 'string' },           // 规范值 schema
  render: (_args, value) => [{ type: 'text', text: value }], // 值 → 模型可见文本
}
```

- **执行器只能返回 `output.schema` 声明的规范 JSON 值**  
（`string / number / boolean / null / array / object`，无损 JSON）。
- `render(args, value)` 把规范值渲染成**模型实际读到的内容**。返回形如 `[{ type: 'text', text: '...' }]` 的数组。
- 可选 `presentationMeta(args, value)`：为顶层直接调用派生 JSON 元数据，随结果持久化、供 UI 回放。

设计理念（DSH 官方「规范输出」契约）：**规范值**（机器可校验、Code Mode 可用、可回放）与**呈现内容**（模型看到的文本）分离。值负责正确性，render 负责可读性。

## 5.3 执行上下文 `exec` 与取消

`execute(args, exec)` 的 `exec` 是 `ToolRunContext`，包含：

|                         |                                 |     |                         |                                                                                                |                                                                     |
|-------------------------|---------------------------------|-----|-------------------------|------------------------------------------------------------------------------------------------|---------------------------------------------------------------------|
| 字段/方法                   | 用途                              | --- | ---                     | <code>exec.signal</code>   只读 <code>AbortSignal</code> ， <b>每个异步工具都必须观测或转发它</b> ，且只能在工作真正停止后结算 | <code>exec.callId</code>   本次调用 id                                  |
| <code>exec.token</code> | 注册表分配的 opaque token（仅用于关联，不跨边界） |     | <code>exec.agent</code> | 发起调用的 agent（可能缺省）                                                                              | <code>exec.session</code>   通过 <code>exec.agent.session</code> 访问会话 |

|| `exec.deferContext(context)` | 把一段上下文推迟到结果抵达循环时注入 ||  
`exec.concludeTurn()` | 结束当前轮次 |

**取消语义（真实契约）：** 取消是协作式的。工具主体调用前发生的取消 →

`ABORTED_BEFORE_DISPATCH`；工具主体被调用后发生的取消 → 只能把成功结果替换为 `ABORTED`。拒绝、包装层失败、工具失败等仍保留更具体的结果。

## 5.4 执行流水线（扩展点全景）

每次工具调用按以下顺序经过（真实契约）：

```
tools/pre-execute    （可重排的 allow/deny/ask 门禁）
  → 已注册的单调 guards (ctx.tools.guard, 拒绝即最终拒绝)
  → tools/execute     （环绕分发包装层：超时/重试/指标；只能替换 signal）
  → 工具主体 execute(args, exec)
  → tools/post-execute （替换 content / 替换 value / 反馈阻止 / 附加上下文）
  → finalizeContent   （工具定义持有的最终内容边界，只能改 content）
  → tools/result       （仅观测的不可变最终结果通知）
```

三个 waterfall 事件是你「横切所有工具」的抓手：

- `tools/pre-execute`：允许/拒绝/询问，**有意不允许改写** `exec.arguments`（否则日志与实际执行脱节）。
- `tools/execute`：包裹分发，用于超时/重试/指标。
- `tools/post-execute`：检查/替换结果、附加上下文。

注意： `timeoutMs` 只是声明（策略元数据），注册表**不会**强制 deadline。要真正强制超时，必须用 `@deepseek-ai/dsh-tool-call-timeout-policy` 这个 `tools/execute` 包装层。

## 5.5 并发： `isConcurrencySafe`

agent loop 会把连续的 `parallel` 调用归入有界滚动池，把 `exclusive` 调用当顺序屏障。工具可声明：

```
isConcurrencySafe: (args) => boolean
```

只有**确切的** `true` 才允许并发分发/主体执行；无效输入和其余情况都是独占。声明并发的工具主体**不得修改父级拥有的状态**；共享状态竞态必须可交换，否则安全拒绝。

决定并发的另一条路：`ctx.tools.executionMode(exec)` 返回 `parallel` 仅当可见定义的分类器恰好返回 `true`；未知/隐藏/未声明/无效/抛错的分类结果均为独占。

## 5.6 工具拥有的 UI 呈现：`presentCall` / `presentResult`

工具可以用纯函数声明自己的 UI 呈现，让界面无需为工具名写特判：

```
presentCall(args)    // 调用视图: { card: 'generic'|'terminal'|'diff', ... }
presentResult(args, result) // 结果视图: { card: 'generic'|'terminal'|'diff'|'sea
```

返回 `undefined` 则用通用回退。呈现器**只依赖参数与持久结果**（UI 会在流式与回放时调用）。`defineTool` 会对旧日志参数做软校验并回退，避免回放崩溃。

## 5.7 案例一：复刻 `todo_write`（最完整的参考实现）

下面是 `@deepseek-ai/dsh-tool-todo` 的精简版，但**结构与真实代码一致**。它演示了：配置驱动描述、会话投影、参数校验、输出契约、UI 呈现。

```
// packages/extensions/tool-todo/src/index.ts (真实结构)
import z from '@deepseek-ai/schemastory'
import { defineTool } from '@deepseek-ai/dsh-tools'

export const name = 'tool-todo'
export const inject = ['tools']
export const Config = z.object({ allowParallelInProgress: z.boolean().required() })

const STATUSES = ['pending', 'in_progress', 'completed']

function describe(allowParallel: boolean): string {
  const head = 'Record and update a structured task list for the current work. Ser
  const mid = allowParallel
    ? 'Mark every todo being actively worked on `in_progress` – several at once wh
    : 'Keep AT MOST ONE todo `in_progress` at a time...'
  const tail = 'Mark a todo `completed` the moment it is done... Statuses: `pendin
  return head + mid + tail
}

export function apply(ctx, config) {
  const allowParallel = config.allowParallelInProgress

  ctx.tools.register(defineTool({
    name: 'todo_write',
    description: describe(allowParallel),
    parameters: {
      todos: {
        type: 'array',
        required: true,
        description: 'The COMPLETE task list, replacing any previous list.',
        items: {
          type: 'object',
          additionalProperties: false,
          properties: {
            content: { type: 'string', required: true, description: 'What the task
            status: { type: 'string', required: true, enum: [...STATUSES],
              description: 'pending (not started) | in_progress (now) | c
          },
        },
      },
    },
    output: {
      schema: {
        type: 'object',

```

```

    additionalProperties: false,
    properties: {
      todos: { type: 'array', required: true, items: { /* ... */ } },
      counts: { type: 'object', additionalProperties: false, required: true,
        pending: { type: 'integer', required: true },
        inProgress: { type: 'integer', required: true },
        completed: { type: 'integer', required: true },
      } },
    },
  },
  render: (_args, value) => [{
    type: 'text',
    text: `Updated todo list: ${value.counts.pending} pending, ${value.counts.
  }],
},
execute(args, exec) {
  // 参数已通过 schema 校验；这里做 schema 表达不了的业务校验
  const todos = toTodoList(args.todos, allowParallel)
  if (!exec.agent) throw new Error('todo_write requires an owning agent session')
  exec.agent.session.append('todo/write', { todos }) // 写会话事件（可回放）
  const count = (s: string) => todos.filter(t => t.status === s).length
  return {
    todos: todos.map(t => ({ content: t.content, status: t.status })),
    counts: { pending: count('pending'), inProgress: count('in_progress'), con
  }
},
presentCall: (args) => ({ card: 'generic', title: 'Update todo list', kind: 'c
}))
}

```

### 值得借鉴的四点：

1. **描述文本由配置驱动**： `allowParallelInProgress` 改变模型行为约束，于是描述文本随之变化——模型看到的指令必须与实际策略一致。
2. **schema 管不了的校验放 execute 里**：如「内容非空、去重、至多一个 `in_progress`」。
3. **副作用写进 session 事件**： `exec.agent.session.append('todo/write', ...)` 使列表可回放、可投影（真实代码还注册了 `sessionProjections` 的 `todos` 单元）。
4. **返回结构化的 counts**：让 `render` 能生成简洁的模型可见文本。



## 5.8 案例二：一个带 HTTP 请求的工具

```
import { defineTool } from '@deepseek-ai/dsh-tools'
import z from '@deepseek-ai/schemastory'

export const name = 'tool-http-get'
export const inject = ['tools']
export const Config = z.object({ baseUrl: z.string().default('') })

export function apply(ctx, config) {
  ctx.tools.register(defineTool({
    name: 'http_get',
    description: 'Perform an HTTP GET request and return the response body as text',
    parameters: {
      url: { type: 'string', required: true, description: 'Absolute or relative URL' },
    },
    output: {
      schema: {
        type: 'object',
        additionalProperties: false,
        properties: {
          status: { type: 'integer', required: true },
          body: { type: 'string', required: true },
        },
      },
    },
    render: (_args, value) => [{
      type: 'text',
      text: `HTTP ${value.status}\n${value.body}`,
    }],
  }),
  async execute(args, exec) {
    const url = new URL(args.url, config.baseUrl || 'http://localhost')
    const res = await fetch(url, { signal: exec.signal }) // 转发取消信号
    const body = await res.text()
    return { status: res.status, body }
  },
  )))
}
```

要点： `fetch` 显式传入 `exec.signal`，让取消能中断在途请求——这正是「每个异步工具都必须观测或转发 `signal`」的落地。

## 5.9 案例三：只读 + 可并发 + 带 read 视图的工具

```

export const name = 'tool-line-count'
export const inject = ['tools', 'fs']

export function apply(ctx) {
  ctx.tools.register(defineTool({
    name: 'line_count',
    description: 'Count the number of lines in a text file.',
    parameters: {
      path: { type: 'string', required: true, description: 'File path' },
    },
    output: {
      schema: {
        type: 'object',
        additionalProperties: false,
        properties: {
          path: { type: 'string', required: true },
          lines: { type: 'integer', required: true },
        },
      },
      render: (_args, value) => [{ type: 'text', text: `${value.lines} lines in ${value.path}` }],
    },
    async execute(args, exec) {
      const target = await ctx.fs.resolve(args.path)
      const text = await ctx.fs.readText(target, exec.signal)
      return { path: args.path, lines: text.split(/\r?\n/).length }
    },
    // 只读、无共享可变状态 → 可并发
    isConcurrencySafe: () => true,
    presentResult: (_args, value) => ({
      card: 'read',
      title: value.path,
      path: value.path,
      lines: [], // 简例；真实 read 视图带行号数组
      totallines: value.lines,
    })),
  }))
}

```

这里展示了三条进阶技巧：**注入** `fs` **服务**（读写走沙箱）、**声明** `isConcurrencySafe`（只读可并发）、**返回** `card: 'read'` **结果视图**。

## 5.10 工具开发检查清单

上线一个工具前，逐项过一遍：

- [] `name` 使用稳定的、模型友好的命名（DSH 惯例是 snake\_case： `read_file` 、 `todo_write` ）。
- [] `description` 精确描述**何时用、怎么用、副作用**，且与实际行为一致。
- [] `parameters` 用 DSL 表达， `required` 与 `description` 齐全；封闭对象显式 `additionalProperties: false` 。
- [] `output.schema` 声明规范 JSON 值；执行器只返回该 schema 允许的值。
- [] `render` 产出简洁的模型可见文本。
- [] `execute` 里转发 `exec.signal` ；做 schema 表达不了的业务校验。
- [] 需要并发的只读工具声明 `isConcurrencySafe: () => true` 。
- [] 有副作用的调用写进 session（供回放/投影）。
- [] 需要超时强制时，配合 `dsh-tool-call-timeout-policy` ，并在 schema 里声明 `timeoutMs` 。

## 5.11 模型体验与 KV Cache 影响

- **普通模式**：模型看到每个可见工具的 `name / description / JSON Schema`。每次请求的固定成本与可见工具数量成正比。
- **KV Cache**：只要可见工具集合与顺序不变，前缀稳定；注册/注销/限制某个工具可能使缓存从第一个变化的 schema token 起失效。
- **Code Mode**： `mode: code|both` 时工具以生成的 `run_code` SDK 呈现，模型用 TypeScript/Python 程序组合多步操作（详见 `dsh-tools` README「Code Mode」）。

## 5.12 本章小结

- 注册: `ctx.tools.register(defineTool({ name, description, parameters, output, execute, ... })))`。
- 契约: 参数 schema 校验 + 输出 schema 强制 + `render` 生成模型文本。
- 执行: `execute(args, exec)`, `exec.signal` 必须协作取消, `exec.agent.session` 写历史。
- 流水线: `pre-execute` → `guards` → `execute` 包装 → 主体 → `post-execute` → `finalizeContent` → `result`。
- 并发: `isConcurrencySafe` 返回 `true` 才并发。
- UI: `presentCall` / `presentResult` 声明工具拥有的呈现。

下一章讲 **Skill**——另一种让模型「学到新能力」的机制。

## 第 6 章 Skill 开发

Skill（技能）是「按需加载的指令/知识」。它和工具互补：**工具给模型「能做什么」，Skill 给模型「怎么做、何时做」**。这一章讲 SKILL.md 的格式、frontmatter、发现根目录、provider 注册，以及如何写一个自定义 skill 提供方。

### 6.1 Skill 是什么，如何进入模型

DSH 的 Skill 体系分三块：

```
| 包 | 角色 | |--|--|| @deepseek-ai/dsh-skill | ctx.skills 注册表 || @deepseek-ai/dsh-skill-filesystem | 本地文件系统提供方（扫描项目/用户目录） || @deepseek-ai/dsh-tool-skill | 面向模型的目录与加载器工具（skill 工具） |
```

模型体验（真实契约）：`dsh-tool-skill` 把可调用的 skill 名称 + 有长度上限的描述渲染到初始/替换目录中，把选中的当前指令正文渲染到保留的工具历史里。**路径、provider rank、被禁用的 skill 对模型隐藏。**

关键点：skill 的 `name / description`（目录概览）与正文（加载后）是**独立的生命周期**——发现阶段解析 frontmatter 生成概览，每次 `skill(name)` 加载时**重新读取文件**，所以正文编辑无需 hash/修订/缓存失效。

### 6.2 SKILL.md 格式

skill 可以是**单层目录 bundle**（`<name>/SKILL.md`）或**平铺 Markdown 文件**（`<name>.md`）。**刻意不支持嵌套** `**/SKILL.md`（发现深度一层）。

格式 = YAML frontmatter + Markdown 正文：

```

---
name: my-awesome-skill
description: 一句话说明这个 skill 何时用、做什么。
whenToUse: 可选，更详细的触发条件。
metadata:
  version: 1
  category: coding
disable-model-invocation: false
user-invocable: true
---

```

# 正文从这里开始

这里是模型加载该 skill 后会读到的完整指令正文.....

## 6.2.1 frontmatter 字段（真实契约）

| 字段                       | 必填 | 含义                                             |
|--------------------------|----|------------------------------------------------|
| name                     | ✓  | skill 名， <b>必须 kebab-case</b> (isSkillName 校验) |
| description              | ✓  | 目录里展示的一句话描述                                    |
| whenToUse                | ✗  | 可选的详细触发条件                                      |
| metadata                 | ✗  | 任意开放对象（目前不授予调用权限）                              |
| disable-model-invocation | ✗  | true 则从模型目录与 loader 排除                         |
| user-invocable           | ✗  | false 则从面向用户命令排除                               |

**调用策略校验遵循「失败时默认拒绝」**：布尔字段用驼峰 (disableModelInvocation) 或非布尔值，都会**记录警告并排除整个 skill**，而不是只丢字段或回退宽松默认。可接受的布尔写法：YAML 布尔值，以及大小写不敏感的 true/false、yes/no、on/off、1/0。

误用驼峰会静默导致你的 skill「不出现」——这是最常见的坑。写 frontmatter 务必用**小写连字符**：disable-model-invocation、user-invocable。

## 6.2.2 正文与资源基底

加载后，resourceBase 是 { kind: 'directory', path } ——即 skill 所在目录。正文里可以用相对路径引用同目录的 references/、scripts/、assets/ 等资源，模型通过文件工具按该基底访问。

## 6.3 发现根目录与 rank

`dsh-skill-filessystem` 按 rank 顺序扫描以下根（真实契约）：

```
| rank | source | 路径 | |---|---|---| | 100 | project-dsh | <projectRoot>/dsh/skills ||
200 | project-agents | <projectRoot>/agents/skills || 300 | custom |
Config.customSkillDirs || 400 | user-dsh | <dshHome>/skills || 500 | user-agents |
<agentsHome>/skills |
```

- **项目根**是「包含 `.git` 的最近祖先目录」；没有 `.git` 则用 `cwd`。
- 用户 DSH 根会跳过 `.system` 子目录（系统拥有的 skill 不当普通用户 skill）。
- **rank 决定同名 skill 的覆盖优先级**：rank 越小优先级越高（project 覆盖 user）。

所以：想给「当前项目」写 skill，放 `<project>/dsh/skills/<name>/SKILL.md`；想给「你自己所有项目」写，放 `~/dsh/skills/<name>/SKILL.md`；想在代码仓库里随项目分发，放 `.agents/skills/`。

## 6.4 写一个 Skill（实战）

目标：写一个「检查并修复 Markdown 链接」的 skill，随项目分发。

### 目录结构

```
<project>/dsh/skills/markdown-link-check/
└─ SKILL.md
```

### SKILL.md

```

---
name: markdown-link-check
description: 检查并修复 Markdown 文档中的死链与相对路径错误。
whenToUse: 当用户要求"检查链接""修复死链""check links"或审查 Markdown 文档时使用。
---

# Markdown 链接检查

## 目标
找出当前仓库 Markdown 文件中的断链。

## 步骤
1. 用 grep 工具列出所有 `](...)` 形式的链接。
2. 对每个相对链接，解析为相对于所在文件的路径。
3. 用 read 或 glob 工具确认目标文件是否存在。
4. 收集所有断链，按文件分组汇报，并给出修复建议（改名/改路径/删除）。
5. 未经用户确认，不要批量改写文件；先给清单。

## 常见陷阱
- 锚点链接（`#xxx`）不视为断链，但可在有目标文件时顺带核对标题。
- 忽略外部 http(s) 链接，除非用户明确要求。

```

把它放进项目后，模型的下一次请求里 `skill` 工具就能看到 `markdown-link-check`，触发词命中时加载正文。

## 6.5 自定义 Skill 提供方（进阶）

如果你想让 skill 来自数据库、远程 API 或其他非文件系统来源，就实现一个 provider 并注册到 `ctx.skills`。

provider 契约（源自 `dsh-skill-filessystem` 的实现，真实结构）：



```
// my-skill-provider.ts
export const name = 'my-skill-provider'
export const inject = ['skills']

export function apply(ctx, config) {
  ctx.skills.registerProvider((control) => {
    return new MyProvider(ctx, control, config)
  })
}

class MyProvider {
  name = 'my-provider'
  constructor(ctx, control, config) {
    // control.signal: 当提供方应停止时触发
    // control.invalidate: 让注册表重新发现（目录变了时调用）
    control.signal.addEventListener('abort', () => { /* 清理 */ }, { once: true })
  }

  // 返回 skill 概览候选 (name/description/whenToUse/rank/provider/source/...)
  async list(options) {
    return candidates
  }

  // 根据候选 locator 加载完整正文, 返回 { name, description, content, resourceBase
  async get(candidate, options) {
    return fullSkill // 文件消失时返回 undefined
  }
}
```

### 关键点:

- `list(options)` 的 `options.cwd` 用于 cwd 敏感的工作区（本地 provider 据此选项目根）。
- `get(candidate, options)` 的 `options.signal` 用于取消文件/网络读取。
- 提供方被 `control.invalidate` 失效后，注册表会重新 `list`。
- 文件系统 provider 还监听 `fs/observed` 事件：第一方 `write / edit` 工具改了可能影响 skill 的文件时，**同步**失效，让模型下一步无需等 watcher 就能看到自己的改动。

## 6.6 目录变更检测（了解即可）

`dsh-skill-filessystem` 用 Chokidar 监视现有 skill 根：只关心**直属 bundle 目录的增删、平铺 .md 文件的增删、SKILL.md 的增删改**；`change` 事件用于重新发现 `name / description` 等 frontmatter。 `references/`、`scripts/`、`assets/` 等资源变更**不会**使目录失效（不影响概览）。

不存在的根会从最近现有祖先开始逐段轮询探测（`fs.watchFile`），所以「创建整个 skills 目录」也能被观察到。

## 6.7 Skill vs 工具：什么时候用哪个

|                  |                   |               |                 |                      |                        |
|------------------|-------------------|---------------|-----------------|----------------------|------------------------|
| 维度               | 工具 (tool)         | Skill (skill) | --- --- ---     | 本质                   | 可执行的函数 + 强类型 schema    |
| 按需加载的指令/知识       |                   | 模型看到          | 每次请求都在 schema 里 | 概览在目录里，正文加载          | 后进历史                   |
| 成本               | 每次请求固定成本（与工具数成正比） | 概览成本低，正文按需加载  |                 | 适合                   | 确定性操作（读文件、执行命令、写 todo） |
| 流程性知识、领域规范、复杂工作流 |                   | 分发            | npm 包（插件）       | 纯 Markdown 文件（放目录即可） |                        |

一句话：「**做一件事**」用工具；「**怎么把一件复杂的事做好**」用 skill。

## 6.8 本章小结

- Skill = frontmatter + Markdown；`name` 必须 kebab-case，`description` 必填。
- 调用策略字段用小写连字符；非法值默认拒绝（整个 skill 被排除）。
- 根目录 rank：project(100/200) > custom(300) > user(400/500)。
- 注册表 `ctx.skills`，提供方实现 `list / get`，通过 `registerProvider` 接入。
- 正文按需加载、每次重读；资源基底指向 skill 目录。

下一章讲 **命令开发**——人类与 Agent 的斜杠命令交互。

## 第 7 章 命令开发

命令 (command) 是**人类**在输入框敲的斜杠命令 (如 `/goal`、`/compact`)。它不经过模型，直接执行一个 handler，并返回 UI 可见的结果。这一章讲 `ctx.commands` 注册表、`CommandResult`、作用域遮蔽，并复刻 `/goal`。

### 7.1 命令注册表： `ctx.commands`

命令插件 `inject: ['commands']`，然后注册：

```
ctx.commands.register({
  name: 'goal', // 不含斜杠
  description: 'set or view the goal for a long-running task',
  input: { hint: '[<objective>|clear|edit <objective>|pause|resume]' }, // 可选输入
  handler: (invocation) => { /* ... */ }, // 返回 CommandResult
})
```

真实约束 (`normalizeDefinition`)：

- `name` 必须匹配 `/^[a-z][a-z0-9_-]*$/`。
- `description` 必须是非空字符串。
- `input.hint` 若非空，必须是字符串。
- `handler` 必须是函数。
- 可选 `recordInput: false`：执行时不把原始输入写进会话日志。

**作用域遮蔽** (真实契约)：普通上下文注册的是**全局**命令；通过「command-injected child of an agent context」注册的只为该 agent 可见，并**遮蔽**同名全局命令。

### 7.2 CommandResult

handler 返回的结果会被 `normalizeResult` 校验：

```
{ kind: 'success', text?: string, sourceEventSeq?: number }
{ kind: 'error', text: string } // error 的 text 必须非空
```

- `sourceEventSeq` 是「结果来自哪个持久化事件」的定位，UI 用于跳转。
- 返回其他 `kind` 或字段类型错误会抛 `TypeError`。

handler 的 `invocation` 参数（真实结构）：

```
{
  commandId: string, // 本次执行的配对 id（写进日志）
  agent: Agent, // 接收该命令的 agent
  rawInput: string, // 斜杠后、命令名后的原始输入
  signal: AbortSignal, // UI 请求的取消信号
}
```

执行生命周期（真实契约）：`command/run` 在 handler 前 append，`command/done` 在结算后 append（成功/错误都记）。语法不匹配或命令未知时**什么都不记**（没进 handler）。

## 7.3 案例：复刻 `/goal`

`@deepseek-ai/dsh-command-goal` 是一个教科书级的命令插件。下面按真实结构精简：

```
// packages/extensions/command-goal/src/index.ts (真实结构)
import { GoalError } from '@deepseek-ai/dsh-goal'

export const name = 'command-goal'
export const inject = ['commands', 'goals'] // 依赖命令注册表 + 目标领域服务

const USAGE = 'Usage: /goal [<objective>|clear|edit <objective>|pause|resume]'

// 解析命令语法：只有这几个控制字；其余任意输入都是 objective
function parseGoalCommand(rawInput: string) {
  const input = rawInput.trim()
  if (input.length === 0) return { kind: 'show' }
  const control = input.toLowerCase()
  if (control === 'clear') return { kind: 'clear' }
  if (control === 'pause') return { kind: 'pause' }
  if (control === 'resume') return { kind: 'resume' }
  if (control === 'edit') return { kind: 'invalid-edit' }
  if (/^edit(?:=\s)/iu.test(input)) return { kind: 'edit', objective: input.slice(4) }
  return { kind: 'create', objective: input }
}

export function apply(ctx) {
  ctx.commands.register({
    name: 'goal',
    description: 'set or view the goal for a long-running task',
    input: { hint: '[<objective>|clear|edit <objective>|pause|resume]' },
    handler: (invocation) => executeGoalCommand(ctx, invocation),
  })
}

function executeGoalCommand(ctx, invocation) {
  const command = parseGoalCommand(invocation.rawInput)
  const current = ctx.goals.get(invocation.agent) // 通过领域服务读当前目标
  switch (command.kind) {
    case 'show':
      return current === undefined
        ? { kind: 'success', text: `No goal is currently set.\n${USAGE}` }
        : renderGoal('Goal', current)
    case 'create':
      // 已有非 complete 目标则拒绝，避免误覆盖
      if (current !== undefined && current.phase !== 'complete')
        return { kind: 'error', text: `A goal is already active. Use /goal edit <objective>` }
      return renderGoal('Goal created', ctx.goals.create(invocation.agent, { objective: command.objective }))
    case 'edit':

```

```

    if (current === undefined) return { kind: 'error', text: `No goal is current` }
    return renderGoal('Goal updated', ctx.goals.edit(invocation.agent, goalRef(current)))
  case 'pause': return renderGoal('Goal paused', ctx.goals.pause(invocation.agent, goalRef(current)))
  case 'resume': return renderGoal('Goal resumed', ctx.goals.resume(invocation.agent, goalRef(current)))
  case 'clear': ctx.goals.clear(invocation.agent, goalRef(current)); return { kind: 'error', text: `Goal cleared` }
}
}

```

### 值得借鉴的三点：

1. **命令只做「解析 + 编排」**：真正的持久化/状态机逻辑在 `ctx.goals` 领域服务里（单点拥有持久化，命令插件很薄）。
2. **返回结构化文本**：`renderGoal` 输出多行 `Status/Rounds/Activation/Commands`，UI 直接展示。
3. **明确的状态守卫**：不同 `phase` 下合法的操作不同，非法操作返回 `{kind:'error'}`，不抛异常。

## 7.4 案例：一个自定义命令 `/ls`

```

export const name = 'command-ls'
export const inject = ['commands', 'fs']

export function apply(ctx) {
  ctx.commands.register({
    name: 'ls',
    description: 'list the current directory',
    handler: async (invocation) => {
      const dir = invocation.rawInput.trim() || '.'
      const target = await ctx.fs.resolve(dir)
      const entries = await ctx.fs.listDir(target)
      const text = entries.map(e => `${e.type} === 'dir' ? 'd' : '-' } ${e.name}`).join(' ')
      return { kind: 'success', text: text || '(empty)' }
    },
  })
}

```

要点：handler 可以是 **async**（真实执行器 `withAbort` 会等待 Promise 并受 `invocation.signal` 控制）；用 `ctx.fs` 而非 `node:fs`，让命令也走沙箱。

## 7.5 命令 vs 工具 vs Skill

|            |         |          |     |         |               |          |         |         |        |      |                   |               |        |      |           |         |         |
|------------|---------|----------|-----|---------|---------------|----------|---------|---------|--------|------|-------------------|---------------|--------|------|-----------|---------|---------|
| 命令 command | 工具 tool | 技能 skill | 触发者 | 人类（输入框） | 模型（tool call） | 模型（按需加载） | 是否进模型历史 | 不（直接执行） | 是（加载后） | 结果去向 | UI（CommandResult） | 模型（render 文本） | 模型（正文） | 典型场景 | 快捷操作、状态查询 | 完成任务的动作 | 领域知识/流程 |
|------------|---------|----------|-----|---------|---------------|----------|---------|---------|--------|------|-------------------|---------------|--------|------|-----------|---------|---------|

## 7.6 本章小结

- `ctx.commands.register({ name, description, input?, handler, recordInput? })`。
- handler 返回 `{ kind: 'success' | 'error', text?, sourceEventSeq? }`，可 `async`。
- `invocation` 提供 `agent / rawInput / signal / commandId`。
- 命令插件应「薄」：解析 + 编排，持久化交给领域服务（如 `ctx.goals`）。
- 全局命令可被 agent 作用域同名命令遮蔽。

下一章讲 **系统提示词**——如何往模型「脑子里的说明书」里注入内容。

## 第 8 章 系统提示词

系统提示词是模型的「说明书」。DSH 把它的组装也做成了插件扩展点：任何插件都能贡献**有序的提示词段**、**工具 schema**、**具名变量与动态上下文**。这一章讲 `ctx.systemPrompt` 的四种扩展方式。

### 8.1 组装注册表： `ctx.systemPrompt`

提供方 `@deepseek-ai/dsh-system-prompt`，服务键 `ctx.systemPrompt`。每次 agent 循环**每步组装一次**，渲染成完整模型提示词。

组装流程（真实契约）：

1. 合并全局层与 `context.scope` 的层；
2. 分离工具 schema；
3. 走 `system-prompt/assemble` waterfall（可协作修改/替换结果）；
4. 恢复一个有效的 `complete` 段为唯一提示词段（若有）；
5. 实施 runtime-context 抑制器（若有）。

渲染（`renderPrompt`）会：插值每个段里的 `{{variable}}`、删除空段、用空行连接。

**未知引用、已注册但无值的引用、格式错误的完整 `{{...}}` 组都会抛错**——明确失败胜过交付格式错误的提示词。

## 8.2 四种扩展方式

### 8.2.1 段： `section`

```
ctx.systemPrompt.section({
  name: 'my-guidance',           // 同层内唯一
  order: 150,                   // 排序（升序拼接）
  text: 'Always prefer X over Y.', // 可含 {{variable}}
  complete: false,              // true 则整段成为完整提示词（覆盖其余所有段）
})
```



排序约定（真实契约）：

- `-100`：harness 身份（`You are an AI agent powered by DeepSeek Harness.`）
- `0`：部署 persona
- `100-199`：工具引导（tool guidance）

多个 `complete: true` 段并存时组装**被拒绝**。一个 `complete` 段会成为最终完整提示词，但 waterfall 得到的上下文/工具/变量保持不变。

**作用域**：在 `agent.ctx` 上贡献的段只为该 agent 可见，并遮蔽同名全局段。

### 8.2.2 动态上下文： `context`

```
ctx.systemPrompt.context({
  name: 'git-status',
  order: 50,
  provide: async (assemble) => ({ text: 'On branch main...' }),
})
```

与段不同，动态上下文在**每次符合条件的组装时求值**，并成为带来源的 runtime-context 快照（user 角色）。适合注入「每步都可能变化」的事实（如当前时间、git 状态、环境变量）。

### 8.2.3 工具 schema： `tools`

```
ctx.systemPrompt.tools((context) => ({
  schemas: [...],           // 限制后的可见集合
  knownNames: [...],        // 限制前的全集（供 toolOrder 使用）
}))
```

实际上 `ToolRuntime` 会**自动**把自己注册为工具提供方——你一般不需要手动做这个，除非要实现自定义工具呈现逻辑。

### 8.2.4 变量： `variable`

```
ctx.systemPrompt.variable('model', (context) => context.agent?.model)
ctx.systemPrompt.variable('cwd', (context) => context.agent?.cwd)
```

变量在段文本里用 `{{name}}` 引用。agent loop 会注册 `{{model}}` / `{{cwd}}`；任何插件都能注册自己拥有的事实（如 `{{date}}`、git 状态）。

## 8.3 案例一：注入部署级 persona

真实代码里，部署 persona 是**唯一由配置提供的提示词片段**，渲染为 order 0 的 `deployment:persona` 段。你可以在 profile 的 `cordis.patch.yml` 里覆盖：

```
- id: system-prompt
  config:
    persona: 'You are a Python backend specialist. Answer in Chinese.'
```

注意 `persona` 是模板：完整的 `{{...}}` 组会按已注册变量插值（`{{model}}` / `{{cwd}}` 可用），**没有字面花括号的转义语法**。空字符串 ⇒ 渲染时删除该段。

### 案例一（代码版）：插件贡献段 + 变量

```
import z from '@deepseek-ai/schemastery'

export const name = 'python-specialist'
export const inject = ['systemPrompt']
export const Config = z.object({ guideUrl: z.string().default('') })

export function apply(ctx, config) {
  ctx.systemPrompt.variable('guideUrl', () => config.guideUrl)

  ctx.systemPrompt.section({
    name: 'python-specialist:guide',
    order: 150,
    text: [
      'You specialize in Python backend work.',
      'Follow the conventions at {{guideUrl}} when available.',
    ].join('\n'),
  })
}
```

## 8.4 案例二：动态上下文（注入当前环境）

```
export const name = 'time-context'
export const inject = ['systemPrompt']

export function apply(ctx) {
  ctx.systemPrompt.context({
    name: 'current-time',
    order: 10,
    provide: () => ({ text: `Current time: ${new Date().toISOString()}` }),
  })
}
```

这与真实的 `@deepseek-ai/dsh-time-context` 同类：把「现在几点」作为带来源的运行时上下文注入，让模型能回答时间相关问题。

## 8.5 案例三： `complete` 段——接管整个提示词

某些部署想**完全接管**系统提示词。这时用 `complete` 段：

```
ctx.systemPrompt.section({
  name: 'full-control',
  order: 0,
  complete: true,
  text: 'You are a strictly controlled assistant. Only answer about topic X.',
})
```

一旦存在一个有效 `complete` 段，它就抑制其他所有段，成为完整提示词；waterfall 得到的上下文/工具/变量不变。**同一组装里出现多个 `complete` 段会抛错**，所以这种「接管」必须由部署方唯一拥有。

## 8.6 模型体验与 KV Cache

- **固定成本**：harness 身份（若启用）、persona、插件段文本，每次请求重复，成本随内容增长。

- **KV Cache**：只要身份、persona、变量、段文本与顺序的渲染结果逐字相同，前缀稳定。任何变更（段文本改一个字、变量值变、顺序变）都可能使缓存从第一个变化的 token 起失效。

**实用含义**：把「每次都会变」的东西放进**动态上下文**（带来源快照）而不是**段里**，能保住系统提示词前缀的缓存稳定性。

## 8.7 常见陷阱

1. **未知变量引用导致组装失败**：`{{foo}}` 里的 `foo` 没注册，`renderPrompt` 抛错。段文本里写花括号要谨慎（无转义语法）。
2. **多个 `complete` 段**：导致组装被拒绝。`complete` 段必须由部署方唯一拥有。
3. **`toolOrder` 配置错误**：列表必须**恰好一个** `'<unlisted-tools>'` 其余项标记；已列名没有对应工具会使每次 `assemble()` 被拒绝（这类错误在**首轮**才暴露，不在启动时）。
4. **把易变内容写进段**：破坏 KV Cache 前缀稳定性。应进动态上下文。

## 8.8 本章小结

- `ctx.systemPrompt.section({name, order, text, complete?})` 贡献段；`order` 约定 -100 身份 / 0 persona / 100-199 工具引导。
- `context` 贡献每步求值的动态上下文；`tools` 贡献 schema（一般自动）；`variable` 贡献 `{{name}}` 变量。
- `complete` 段接管整段提示词；多个并存抛错。
- 组装走 `system-prompt/assemble` waterfall 后渲染；未知变量抛错。
- 稳定内容进段，易变内容进动态上下文。

下一章讲 **Agent Preset**——如何把插件按「每个会话」的作用域组合成可选的 Agent 形态。

## 第 9 章 Agent Preset 与作用域组合

Agent Preset（预设）是 DSH 最精巧的机制之一：它把一组插件按「每会话」的作用域组合成一种可选的 Agent 形态（如「标准模式」「PTC 模式」），且多个会话共享同一个「常驻组合」而不互相污染状态。这一章讲 `preset.yml`、`agent.cordis.yml`、`cordis:group` 与 `isolate realm`。

### 9.1 两个平面：host-plane 与 agent-plane

理解 Preset 前，先分清两个平面：

| 平面                         | 作用域 | 内容                    | 谁拥有                                                         |
|----------------------------|-----|-----------------------|-------------------------------------------------------------|
| <b>host-plane</b> (宿主)     | 进程级 | 注册表本身、沙箱/审批栈、持久化、模型路由 | bundle ( <code>dsh-base</code> 、 <code>dsh-web-app</code> ) |
| <b>agent-plane</b> (Agent) | 每会话 | 工具、提示词段、每会话状态         | <code>agent.cordis.yml</code> (preset)                      |

真实代码 `config/agent-presets/standard/agent.cordis.yml` 的头部注释说得非常清楚：

宿主组合 ( `base.cordis.yml` + `web.cordis.yml` ) 保留一切 preset 不该拥有的东西：注册表本身、沙箱与审批栈、持久化、模型路由。

**判断某行属于哪个平面的一条准则**（真实注释反复强调）：「**宿主行会 inject 一个服务**」就是**宿主平面的判据**——注入在会话存在之前解析，没有 agent 可键控；而 agent-plane 行只「注册进」宿主注册表、不提供需要宿主注入的服务。

### 9.2 目录结构与两份文件

一个 preset 是一个目录：

```
config/agent-presets/<id>/
├─ preset.yml          # 显示名、描述、排序
└─ agent.cordis.yml    # agent-plane 插件组合（真正的"配置树"）
```

真实 `preset.yml` （ `standard` ）：

```
name: 标准模式
description: 功能完整的编码 Agent，支持文件编辑、Shell、文件与网页检索、Skills、计划、
order: 1
```

`order` 决定选择器里的排序。 `@deepseek-ai/dsh-agent-presets` 通过配置的 `roots`（信任级别 `system`）发现这些目录；启动器把随附的 `config/agent-presets/` 注入为 `system root`。

## 9.3 `agent.cordis.yml` 的写法

`agent.cordis.yml` 与 `cordis.patch.yml` 结构相同（`- insert:` 列表 + `- id:` 覆盖），但语义不同：它是 `agent-plane` 组合，由 `roster` 每个进程挂载一次在一个「常驻作用域」下，每个命名该 `preset` 的会话按 **scope 父子关系 join** 进来。

### 9.3.1 普通行（无需 realm）

大多数工具行只是「注册进宿主注册表」，不需要 `realm`。真实

`standard/agent.cordis.yml` 里：

```

- id: tool-fs
  name: '@deepseek-ai/dsh-tool-fs'

- id: tool-fs-search
  name: '@deepseek-ai/dsh-tool-fs-search'
  config:
    sampleOverCapGlobResults: false

- id: skill-filessystem
  name: '@deepseek-ai/dsh-skill-filessystem'

- id: tool-skill
  name: '@deepseek-ai/dsh-tool-skill'

- id: tool-todo
  name: '@deepseek-ai/dsh-tool-todo'
  config:
    allowParallelInProgress: true

```

这些行只注册进宿主 `tools` 注册表、不 `provide` 任何东西，所以不需要 `realm`（它们按 `session` 键控状态，宿主单实例服务所有会话）。

### 9.3.2 需要 `realm` 的行: `cordis:group` + `isolate`

当一个服务不能跨会话共享（比如 `plan-mode` 状态是每 `agent` 的），就必须放进一个带 `isolate` `realm` 的 `group`。真实 `standard/agent.cordis.yml` 的 `plan-mode` 段：

```

- id: planning
  name: cordis:group          # 分组插件
  group: true
  isolate:
    planMode: true          # 为 planMode 服务开辟 entry-local realm
  config:
    - id: plan-mode
      name: '@deepseek-ai/dsh-plan-mode'
      config:
        section: |
          You are in plan mode. Stay in plan mode until exit_plan_mode succeed

```

`cordis:group` 是 `@deepseek-ai/cordis-plugin-group` 提供的**嵌套插件组**：`group: true` 表示这一行本身是分组，`config` 里是子行列表；`isolate`：映射「服务名 → realm 标签」。

真实注释对 `isolate` 的语义说得极准：

一个服务行**必须**坐在带 `isolate realm` 的 `group` 里。没有它，服务发布进 `root realm`，就是进程级——另一个 `preset` 发布同名会冲突，宿主读者会为每个会话解析到同一个 `preset` 实例；`dsh-agent-presets` 在挂载时拒绝这一点。`true` 表示 **entry-local realm**：这个常驻挂载自己的私有实例，与其他 `preset` 隔开。（共享标签不会池化实例——`provide()` 在第二个 `realm` 符号下注册会抛错；标签 `join` 的是 `REALM`，不是这个文件需要的。）

所以 `isolate: { planMode: true }` 里的 `true` 是「每个预设一份独立实例」的开关。

### 9.3.3 更多 realm 例子

真实 `standard` 里 `compaction` 与 `delegation` 也各自开了 `realm`：

```
- id: compaction
  name: cordis:group
  group: true
  isolate:
    compaction: true
    toolResultPruner: true
  config:
    - id: compaction-basic
      name: '@deepseek-ai/dsh-compaction-basic'
    - id: command-compact
      name: '@deepseek-ai/dsh-command-compact'
    - id: tool-result-pruner
      name: '@deepseek-ai/dsh-compaction-tool-result-pruner'
      config: { thresholdChars: 8192, headChars: 4096, tailChars: 1024 }
```

注释解释为什么 `compaction` 需要 `realm`、而 `token-meter` 不需要：



`compaction-basic` 用 `ctx.get` 读 `toolResultPrune`，所以 `pruner` 必须共享这个 realm。..... `tokenMeter` 刻意不在这个 realm：meter 留在宿主平面，这里的行解析宿主那一份实例。它每 session 键控折叠、拥有浏览器读取的投影单元——放进 realm 会让这些单元随 preset 挂载与否而来去。

## 9.4 实战：复制 standard 做定制 preset

目标是做一个「极简 preset」，只给 Agent 文件工具 + skill，去掉 shell。

### 第 1 步：复制目录

```
cp -r config/agent-presets/standard config/agent-presets/minimal-custom
```

### 第 2 步：改 `preset.yml`

```
name: 极简模式
description: 只有文件读写与 Skill，无 Shell 与网络。
order: 3
```

### 第 3 步：裁剪 `agent.cordis.yml`

删掉 `tool-bash`、`tool-pwsh`、`tool-web` 等行（或对相应行写 `disabled: true`）。保留文件工具与 skill。

### 第 4 步：让 roster 能发现它

随附 preset 根是 system root；自定义 preset 通常放在用户可写的 root。启动器把 `config/agent-presets/` 作为 system root 注入（见 `profile-boot` 里对 `agent-presets` 行的处理）。你的部署如果配置了可写 root（`dsh-agent-presets` 的 `copy / remove` 能力），就能在 UI 里复制/删除 preset。

真实能力： `ctx.agentPresets` 提供

`list/resolve/mount/composeFrom/recompose/read/copy/remove` 等（见 `dsh-tool-cordis` 的 API 目录）。「复制」是**唯一**的作者化写入：按 id 复制整个目录，不验证挂载（源今天能挂载，副本今天就能挂载）。

## 9.5 挂载与 join：会话如何「使用」一个 preset

- **mount**：agent 工厂的 `setup(agentCtx)` 调用 `ctx.agentPresets.mount(agentCtx, id)`，确保 preset 的「常驻组合」存在，再把 agent 的 scope 键 parent 到它，让挂载的注册/监听覆盖这个 agent。
- **composeFrom (join)**：子 agent 通过 `composeFrom(agentCtx, parentCtx)` **绑定**到父 agent 已有的同一份常驻组合——**是绑定而非重新挂载**，所以子 agent 拿到的是父 agent 的**同一实例**（同一插件对象、同一工具注册、同一提示词段）。这保证「父子能力继承」不因 preset 文件被编辑而产生代际差异。

真实代码里，两个 in-process subagent 驱动在**同步的** `setup` 里用 `composeFrom` 组合子 agent，因为它是同步、无组合失败模式的。

## 9.6 常见陷阱

1. **把需要 realm 的服务行放 group 外**：`dsh-agent-presets` 挂载时**拒绝**（会发布进 root realm，进程级污染）。
2. **误以为共享 label 能池化实例**：`isolate` 的 label join 的是 **realm**，不是实例池；同一 realm 下二次 `provide` 抛错。
3. **宿主平面行放进了 preset**：宿主注册表（jobs、subagents、token-meter 等）必须在宿主平面，preset 只该贡献「工具/提示词段」。判断准则：宿主行会 `inject` 服务。
4. **改了 preset 文件后子 agent 拿到不同代际**：join 用 `composeFrom` 绑定父实例，不重读 roster；只有新 mount 才重读。

## 9.7 本章小结

- 两个平面：host-plane（进程级，bundle 拥有）vs agent-plane（每会话，preset 拥有）。
- preset = preset.yml（元数据）+ agent.cordis.yml（agent-plane 组合）。
- 需要跨会话隔离的服务用 cordis:group + isolate: { svc: true } 开 entry-local realm。
- 宿主注册表单例服务所有会话；preset 只贡献工具/提示词段。
- 子 agent 通过 composeFrom join 父实例，保证能力继承的代际一致。

下一章讲 **客户端插件**——把扩展能力带到浏览器界面。

## 第 10 章 客户端插件

除了宿主（Node 侧）插件，DSH 还能加载**浏览器侧**插件——扩展 Web 界面的 UI（设置面板、侧边栏、消息呈现等）。这一章讲 `dsh.client` manifest、客户端 bundle、slot 系统与加载机制。

### 10.1 双面包：一个包，两半代码

客户端插件是一个「双面（dual-face）」npm 包：

- **Node 半**：宿主侧（可能空的 `apply`，或提供宿主服务）；
- **浏览器半**：`exports["./client"]` 指向**构建后的客户端 bundle**，由浏览器加载并执行其 `apply`。

真实例子 `@deepseek-ai/dsh-cordis-client-runner` 的 Node 半是**空** `apply`：

```
// dsh-cordis-client-runner/lib/index.js（真实代码）
/** Host plugin body – this package contributes nothing host-side. */
function apply() {}
export { apply }
```

它存在的意义只是「让这一行出现在宿主 `cordis.yml` / Loader 里」，真正的能力在浏览器半（`exports["./client"]`）——由 `dsh.client` 声明发现。

### 10.2 `dsh.client` manifest

客户端插件在 `package.json` 里声明：

```
// @deepseek-ai/dsh-client-ui-settings-plugins/package.json (真实节选)
{
  "name": "@deepseek-ai/dsh-client-ui-settings-plugins",
  "exports": {
    ".": { "types": "./lib/types/index.d.ts", "default": "./lib/index.js" },
    "./client": { "types": "./lib/types/client/index.d.ts", "default": "./lib/client/index.js" },
  },
  "dsh": {
    "client": {
      "inject": [
        "@deepseek-ai/dsh-client-connection",
        "@deepseek-ai/dsh-client-locale",
        "@deepseek-ai/dsh-client-runtime",
        "@deepseek-ai/dsh-client-ui-settings",
        "@deepseek-ai/dsh-api-remotes"
      ],
      "platform": "web"
    }
  }
}
```

字段（真实校验 `parseDshClient`）：

| 字段                       | 含义                       |
|--------------------------|--------------------------|
| <code>platform</code>    | 目标平台（web 等），不符合当前平台的包被跳过 |
| <code>inject</code>      | 客户端依赖的服务包名列表（字符串数组）      |
| <code>immediately</code> | 可选布尔，控制是否立即加载            |

还必须提供 `exports["./client"]`（字符串或带 `default` 的条件对象）。缺失会报 `declares dsh.client but exports no "./client" bundle`。

## 10.3 客户端 bundle 与加载

客户端 bundle 是**构建产物**（包内 `scripts` 里是 `"bundle": "tsdown"`），产物形如：

```

window.__ModuleLoader__.load({
  id: "@deepseek-ai/dsh-client-ui-settings-plugins",
  factory: (require) => {
    // ... require("@deepseek-ai/dsh-client-ui-slots") 等
    // 最终:
    exports.apply = apply
    exports.inject = inject
    return module.exports
  }
})

```

加载链路（真实契约）：

1. 宿主 `dsh-client-modules` （ `ctx.clientModules` 服务）**增量扫描** Loader 条目里声明了 `dsh.client` 的包；
2. 组合出 `window.__DSH_BOOT__` 引导清单（ `graph()` ），并 `tapIndex` 注入到 HTML；
3. 通过 `/plugins/<id>/client.js?rev=<hash>` 路由提供 bundle；
4. 浏览器按 manifest 的 `inject` 加载客户端服务依赖，再加载 bundle，执行 `apply(ctx)`。

**HMR:** `dsh-client-modules` 监听 bundle 变化，re-hash 后更新 rev，`onRebuilt` / `onGraphChanged` 通知前端刷新（这就是「client-plugin HMR receiver」的由来）。

## 10.4 客户端插件的 `apply(ctx)`

客户端 `apply` 也是一个 Cordis 插件，只是运行在浏览器的客户端 Cordis 运行时里。它能用的客户端服务（真实代码 `dsh-client-ui-settings-plugins` 的 `apply`）：

```
function apply(ctx) {
  const { api } = ctx.get('connection') // 连接服务 (API 客户端)
  const t = ctx.locale.bind(NS) // 国际化
  ctx.effect(() => ctx.locale.register(NS, { zh, en }), 'dictionaries') // 注册文

  // 用 slot 系统注入 UI: 在 "settings.section" 插槽里注册 "plugins" 设置节
  ctx.slots.inject('settings.section', () => ctx.slots.register({
    name: 'settings.section',
    id: 'plugins',
    order: 15,
    label: () => t('nav'),
    locale: NS,
    inject: sectionInjected,
    children: { 'settings.plugins.tab': { kind: 'list', scope: 'root' } },
  }, PluginsSettingsSection))

  // 注册子项 (generator 形式可 yield 多个 register)
  ctx.slots.inject('settings.plugin.item', function* () {
    yield ctx.slots.register({ name: 'settings.plugin.item', id: 'bash', order: 0,
    yield ctx.slots.register({ name: 'settings.plugin.item', id: 'agent-loop', order: 1,
  })

  // 订阅服务端事件
  ctx.effect(() => ctx.remote.$on('credentials/updated', (ref) => { /* ... */ })),
}
```

## 关键客户端服务：

| 服务 | 用途 | --- | --- | | `ctx.slots` | UI 插槽系统：

`register` / `inject` / `entries` / `subscribe` / `getVersion` || `ctx.locale` | 国际化：

`bind` / `register` / `getSnapshot` / `subscribe` || `ctx.get('connection')` | 到宿主的连接 ( `api` 客户端) || `ctx.remote` | 服务端事件的远程订阅 ( `$on` ) ||

`ctx.settingsScope` | 设置作用域读写 |

## 10.5 Slot 系统：客户端插件的扩展点

Slot 是客户端 UI 的插槽机制，让插件把 React 组件挂进界面的命名位置：

```
ctx.slots.register(name, options, Component) // 注册一个 slot 条目
ctx.slots.inject(name, setup)                // 声明某 slot 的子结构（list/嵌套）
ctx.slots.entries(name)                     // 读某 slot 的所有条目
```

`options` 里的 `id`（条目身份）、`order`（排序）、`label`（文案）、`locale`（字典命名空间）、`inject`（子结构快照 getter）是常见字段。真实代码里 `settings.section` → `settings.plugins.tab` → `settings.plugin.item` 就是一层层嵌套的 slot 树。

**这意味着：**要往 Web 界面加一个自定义设置面板，就注册一个 `settings.section` 的 slot 条目（或更深层的 slot），配上 React 组件即可。

## 10.6 一个最小客户端插件（骨架）

```
// package.json
{
  "name": "@you/ui-hello",
  "type": "module",
  "exports": {
    ".": "./lib/index.js",
    "./client": "./lib/client.js"
  },
  "dsh": {
    "client": {
      "inject": ["@deepseek-ai/dsh-client-ui-slots"],
      "platform": "web"
    }
  },
  "peerDependencies": {
    "@deepseek-ai/cordis": "^4.0.1",
    "@deepseek-ai/dsh-client-ui-slots": "*",
    "react": "^18.2.0"
  }
}
```

```
// lib/index.js — Node 半（空）
export function apply() {}
```



```
// src/client.tsx — 浏览器半（构建成 lib/client.js）
export function apply(ctx) {
  ctx.slots.register(
    'settings.section',
    { id: 'hello', order: 99, label: () => 'Hello' },
    HelloSection,
  )
}
```

装进 profile 后，浏览器端就会多一个「Hello」设置节。

完整 UI 插件还需要 locale 字典、settingsScope 读写、React 组件与 CSS 打包（参考 `dsh-client-ui-settings-plugins` 用 `tsdown` + CSS modules 的构建方式）。细节超出本书范围，但**加载机制**已经清晰：manifest 声明 → 宿主扫描 → 浏览器执行 `apply` → slot 注入 UI。

## 10.7 宿主插件 vs 客户端插件：何时用哪个

| 宿主插件 (Node)             | 客户端插件 (Browser)   | 运行环境                      | Node 进程                  | 浏览器                                         |
|-------------------------|-------------------|---------------------------|--------------------------|---------------------------------------------|
| 扩展什么                    | 服务/工具/命令/提示词/模型能力 | UI (面板/卡片/呈现)             | manifest                 | 无 (靠 <code>dsh.bundle</code> 或直接列在 patch 里) |
| <code>dsh.client</code> | 产物                | <code>lib/index.js</code> | ( <code>apply</code> 导出) | <code>exports["./client"]</code>            |
| 构建 bundle               | 依赖注入              | <code>ctx.inject</code>   | <code>ctx.provide</code> | <code>dsh.client.inject</code>              |
| 客户端服务                   |                   |                           |                          |                                             |

两者可以共存于**同一个包**（Node 半提供服务，浏览器半提供对应 UI）。

## 10.8 本章小结

- 客户端插件是双面包：`dsh.client` manifest + `exports["./client"]` 构建产物。
- 宿主 `dsh-client-modules` 扫描、组合 `window.__DSH_BOOT__`、经 `/plugins/<id>/client.js` 提供，并支持 HMR。

- 浏览器 `apply(ctx)` 用 `ctx.slots` (UI 插槽)、`ctx.locale`、`ctx.get('connection')`、`ctx.remote` 等客户端服务。
- Slot 系统是 UI 扩展点: `register` / `inject` / `entries` 组成嵌套槽树。

下一章讲**安全、沙箱与审批**——插件作者如何安全地接入副作用。

## 第 11 章 安全、沙箱与审批

插件（尤其是工具插件）常常要「做有副作用的事」——读写文件、执行命令、发网络请求。DSH 用**沙箱 + 审批 + 凭据**三层机制约束这些副作用。这一章从**插件作者视角**讲清楚怎么安全地接入。

### 11.1 三件套总览

真实 `dsh-base/cordis.patch.yml` 里，安全相关的行（节选）：

```
- id: sandbox
  name: '@deepseek-ai/dsh-sandbox-local'

- id: sandbox-policy
  name: '@deepseek-ai/dsh-sandbox-policy'
  config:
    mode: !!js process.env.DSH_PERMISSION_MODE ?? 'workspace-write'
    workspaceRoot: !!js process.cwd()

- id: approval
  name: '@deepseek-ai/dsh-user-approval'
  config:
    policy: !!js "(process.env.DSH_PERMISSION_MODE ?? 'workspace-write') === 'dang

- id: permission
  name: '@deepseek-ai/dsh-permission-presets'
  config:
    presets:
      read-only:      { sandbox: read-only, approval: ask }
      workspace-write: { sandbox: workspace-write, approval: ask }
      danger-full-access: { sandbox: danger-full-access, approval: never }

- id: credentials
  name: '@deepseek-ai/dsh-credentials-local'
```

三个机制：

| 机制 | 服务 | 作用 | |---|---|---| | **沙箱** | `sandbox / sandbox-policy` | 限制文件系统/进程能碰到哪 (`read-only / workspace-write / danger-full-access`) | | **审批** | `approval / permission` | 对危险操作先问用户 (`ask/never`) | | **凭据** | `credentials` | 解析 API key 等秘密, 不把它们写进进程环境 |

---

## 11.2 沙箱模式

`DSH_PERMISSION_MODE` 环境变量 (默认 `workspace-write`) 决定沙箱模式:

| 模式 | 含义 | |---|---| | `read-only` | 只读: 只能读文件, 不能写 | | `workspace-write` | 可写工作区 (`workspaceRoot`, 默认 `process.cwd()`) | | `danger-full-access` | 完全访问 (审批也关闭) |

平台沙箱后端按平台挂载: `dsh-bash-sandbox` (非 Windows)、`dsh-pwsh-sandbox` (Windows), `disabled` 用 `!!js process.platform === 'win32'` 表达式动态决定 (第 3 章见过)。

**插件作者要点:** 你的工具如果要做文件/命令副作用, 应该:

1. 通过 `ctx.fs` **服务读写文件** (而非 `node:fs`) —— `fs` 服务是「抽象文件系统提供方」, 沙箱后端会在写操作时按 `sandboxPolicy` 围栏。
  2. 真正需要提权时, 通过审批流程, 而不是绕过沙箱。
- 

## 11.3 审批: `ctx.approval`

审批服务 (`ctx.approval`) 是**可选 seam**——工具注册表在可用时用它, 不可用时把「询问」退化为「拒绝」(`fail-closed`)。这就是为什么 `dsh-tools` 里审批的消费是 `ctx.get('approval')` 而非静态 `inject`。

审批策略词汇:

- `ask`: 问用户 (默认)
- `never`: 不问 (危险模式)
- `allowed-once`: 单次放行 (`request()` 返回的唯一 `grant`)

**工具执行流水线里的审批：** `tools/pre-execute` 可返回 `{kind: 'ask'}`；挂载 `ctx.approval` 时由它处理，否则退化为拒绝。

插件作者一般不需要直接调审批 API，而是**依赖工具流水线 + 沙箱**。但如果你写的是一个「横切工具」的 `tools/pre-execute` 门禁，就要理解这个语义：

```
// 伪代码：一个 pre-execute 门禁，对危险工具名要求 ask
ctx.on('tools/pre-execute', (exec) => {
  if (DANGEROUS_TOOLS.has(exec.name)) {
    return { kind: 'ask', reason: 'this tool mutates the repository' }
  }
  return { kind: 'allow' }
})
```

`PreToolDecision` 三种形态（真实契约）：`{kind:'allow'}` | `{kind:'deny', reason}` | `{kind:'ask', reason?}`。注意它**有意不提供输入改写**。

## 11.4 凭据： `ctx.credentials`

凭据服务（`ctx.credentials`）抽象了「秘密从哪来、怎么读写」。四个操作（真实契约）：

| 方法 | 用途 | |---|---| | `resolve(ref)` | 解析一个凭据引用为当前值（每次调用重新解析，不跨操作缓存） | | `describe(ref)` | 描述配置状态（不暴露值） | | `set(ref, value)` | 持久化存储（拒绝空值、拒绝被只读源遮蔽时写） | | `unset(ref)` | 移除 |

**一条 seam 级规则：**空存储值在所有地方都是「不存在」——`resolve` 跳过它、`describe` 报告未配置，所以空字符串永远不能伪装成已配置的秘密。

真实 `dsh-llm-deepseek` 的 key 就通过凭据存储解析，**不把 key 写进进程环境**：

```
- id: web-search-deepseek
  name: '@deepseek-ai/dsh-web-search-deepseek'
  config:
    apiKeyEnv: DEEPSEEK_API_KEY    # 引用的是"凭据引用"，按请求解析
```

插件作者要点：**不要在插件里硬编码或手写环境变量读 key**，用

`ctx.credentials.resolve(ref)`，让部署方决定秘密从环境、

`$DSH_HOME/.credentials.yaml` 还是项目 `.env` 来。

## 11.5 文件系统服务： `ctx.fs`

`ctx.fs` 是「抽象文件系统提供方」（真实 `dsh-fs-local` / `dsh-fs-sandbox`）。它是插件读写的正确入口，因为：

1. 目标身份稳定（`resolve` 返回 stable `target`，同一文件同一 `targetKey`）；
2. 读暴露 UTF-8 文本或类型化错误，列表稳定且不含内容；
3. 写是**原子**的（`writeText` / `editText`，可带版本守卫防陈旧）；
4. 沙箱后端在写时按 `sandboxPolicy` 围栏。

核心 API 速览（真实契约）：

```
await ctx.fs.resolve(path)           // → FsTarget
await ctx.fs.readText(target, signal) // → string
await ctx.fs.streamText(target, signal) // → AsyncIterable<string>
await ctx.fs.listDir(target, signal)  // → FsDirEntry[]
await ctx.fs.writeText(target, content, expected?, signal?, sandboxPolicy?)
await ctx.fs.editText(target, edit, expected?, signal?, sandboxPolicy?)
await ctx.fs.stat(target, signal)     // → FsInfo | undefined
```

第 5 章案例三已经演示了 `ctx.fs.resolve` + `readText`。要点：**相对路径解析、UTF-8 校验、原子写、版本守卫**都由 `fs` 服务统一承担，插件作者不必重复实现。

## 11.6 权限预设：一个动作切三档

`dsh-permission-presets` 提供三个预设（`read-only` / `workspace-write` / `danger-full-access`），每个预设同时改 `sandbox` 与 `approval` 两个维度。UI 的「权限预设」选择器就是切这个。插件作者：

- 不需要关心预设怎么切；

- 只需要保证：在 `read-only` 下你的工具不会崩溃（写操作应返回清晰的 `FS_*` 类型化错误，而不是未处理异常）；
- 在 `danger-full-access` 下不会越界（不要假设审批永远在，`never` 时你的副作用会直接执行）。

## 11.7 常见陷阱

1. 用 `node:fs` 绕过沙箱：插件作者若直接用 `node:fs` 读写，就绕过了 `ctx.fs` 的沙箱围栏。写文件/命令副作用务必走 `ctx.fs`。
2. 把审批当「总是 ask」：`danger-full-access` 下 approval 是 `never`，`ctx.get('approval')` 也可能缺省。要 fail-closed，把「询问」在无审批时退化为「拒绝」。
3. 空值伪装秘密：`credentials.set(ref, '')` 会被拒绝；`resolve` 跳过空值。别指望空字符串能「清空」凭据（用 `unset`）。
4. 缓存凭据值：`credentials.resolve` 是每次调用重新解析的，插件不要跨操作缓存，否则改凭据后不生效。

## 11.8 本章小结

- 沙箱（`read-only` / `workspace-write` / `danger-full-access`）限制副作用范围，由 `DSH_PERMISSION_MODE` 决定。
- 审批（`ask` / `never`）是可选中 seam, fail-closed；`tools/pre-execute` 返回 `{kind:'ask'}` 触发。
- 凭据（`ctx.credentials`）解析秘密，空值即不存在，不写进程环境。
- 文件副作用走 `ctx.fs`（原子写、版本守卫、沙箱围栏）。

下一章是最后一章——**调试、热更新与发布**，把插件送上生产线。

## 第 12 章 调试、热更新与发布

插件写完了，怎么验证、调试、热更新、并发布给别人用？这一章覆盖从「本地跑起来」到「发布 npm 包」的完整闭环。

### 12.1 检查配置树： `--dump-config`

不动手 boot 进程，先看「组合后的树长什么样」：

```
# 含用户层与 --patch
dsh --profile <name> --dump-config

# 只打印 bundle 层（不含用户层），用于恢复诊断
dsh --profile <name> --dump-default-config
```

用途：

1. **确认插件行真的进去了**：看你的 bundle 是否被对账进 `dsh.profile.bundles`、`patch` 是否正确。
2. **确认覆盖生效**：看你写的 `config` 是否覆盖了 bundle 默认值。
3. **确认 `!!js` 表达式的求值结果**：`disabled` 是否如预期。

### 12.2 诊断「插件没生效」

插件「没反应」几乎总是两种原因：**没激活**（依赖未满足）或**启动失败**（fiber 进入 error 态）。

排查顺序：

1. **看加载是否成功**：启动日志/激活审计会报 `FAILED` fiber。Cordis 的 `fiber.await()` 会重抛启动错误（配置校验错误 `ValidationError`、插件 `apply` 抛错）。
2. **看依赖是否满足**：`inject` 的服务没被提供 → 插件停在 `inactive`。用 `--dump-config` 确认「提供服务的那行」也在组合里。



3. **看状态**: `fiber.getEffects()`、`ctx.registry` 可遍历已注册插件与 fiber。 `dsh-tool-cordis` 甚至把「检查运行时」做成了模型可用工具 ( `cordis_inspect` ) 。
4. **看日志**: `ctx.logger` 内建带名字的 logger; 插件里 `ctx.logger.warn/error` 会带上 fiber 名。

常见信号:

| 信号                                                               | 含义                                                |
|------------------------------------------------------------------|---------------------------------------------------|
| <code>ValidationError: invalid config:</code>                    | <code>Config schema</code> 校验失败 (会列出每条 issue 的路径) |
| <code>service "x" has been registered at &lt;fiber&gt;</code>    | 同名服务重复提供                                          |
| <code>cannot get required service "x" in inactive context</code> | 访问了未注入/不可用的服务                                     |
| <code>INACTIVE_EFFECT</code>                                     | 在已 dispose 的 fiber 上创建 effect                     |
| <code>declares no dsh.bundle</code>                              | bundle 名写进列表但包没声明                                 |
| <code>dsh.bundle.patch</code>                                    |                                                   |

## 12.3 热更新 (HMR)

DSH 的 patch 层是**热更新**的: 长生命周期的表面 (web) 会 watch 用户 patch 文件, 改了就重组合。

真实机制 ( `@deepseek-ai/dsh/lib/profile-boot-*.js` 的 `watchUserPatches` ) :

- 启动时若没有 `hmr` 服务, 会先装 `@deepseek-ai/cordis-plugin-hmr` ;
- watch profile 的 `cordis.patch.yml` 与 home 级 `cordis.patch.yml` ;
- 变更时按新的 patch 栈重新组合, 通过 `fiber.update()` (走 `internal/update waterfall`) 让受影响插件卸载/重启。

这意味着: 改 `cordis.patch.yml` 里的配置, **web 界面下通常无需重启进程**。但注意:

- **改插件代码** (你的 `lib/index.js` ) 不等于改配置——HMR 的入口是 patch 文件, 不是源码文件。源码改动要重新构建 + 重启 (或配合客户端 HMR, 见下) 。
- **客户端 HMR**: `dsh-client-modules` 监听客户端 bundle 变化, re-hash 后更新 rev 并通知前端。但「client-plugin 改动不刷新即可重载」只在你同时跑 `pnpm run dev:web` 重建 bundle 时才成立 (这是本书所在 GUI 的明确限制) 。

## 12.4 日志

插件里：

```
ctx.logger.info('doing %s', thing)    // 命名 logger，自动带 fiber 名
ctx.logger.warn(...)
ctx.logger.error(error)               // Error 会被格式化（含 cause/AggregateError）
ctx.logger.debug(...)
```

`ctx.logger()` 创建带名字的 logger；`ctx.logger.info(...)` 直接用当前 fiber 派生名。可 `ctx.logger.exporter(...)` 注册自定义 sink。

## 12.5 发布：从本地包到 npm

### 12.5.1 发布一个宿主插件（bundle 或普通插件）

标准 npm 包。bundle 需满足：

```
{
  "name": "@you/my-bundle",
  "type": "module",
  "main": "lib/index.js",
  "exports": {
    ".": "./lib/index.js",
    "./cordis.patch.yml": "./cordis.patch.yml"    // bundle 才需要
  },
  "dsh": { "bundle": { "patch": "./cordis.patch.yml" } },    // bundle 才需要
  "peerDependencies": { "@deepseek-ai/cordis": "^4.0.1" }
}
```

插件本体导出 `name` / `inject` / `Config` / `apply`（loader 约定）。

### 12.5.2 发布客户端插件

额外满足 `dsh.client` + `exports["./client"]`（见第 10 章）。

### 12.5.3 安装进 profile

```
dsh plugin --profile mypro add @you/my-bundle
```

## 12.5.4 git 托管插件的构建脚本

真实陷阱 ( `@deepseek-ai/dsh/lib/plugin-*.js` ) :

```
// git-hosted plugins build on install via their prepare script, which pnpm
// blocks until allowed – add the exact key pnpm printed above under allowBuilds
```

`dsh plugin add git+https://...` 时, pnpm 默认阻止依赖的 `prepare` 构建脚本。报错后, 把 pnpm 打印的 **key** 加到 profile 的 `pnpm-workspace.yaml` :

```
packages:
  - .

nodeLinker: hoisted
autoInstallPeers: false

allowBuilds:
  - '@you/my-bundle'    # ← 用 pnpm 提示的确切 key
```

然后重跑。

## 12.6 发布前检查清单

- [] `package.json` 有 `type: "module"`, `main` / `exports` 正确。
- [] bundle 声明了 `dsh.bundle.patch`, 且 `cordis.patch.yml` 可被 `--dump-config` 正确解析。
- [] 插件导出 `name` / `inject` / `Config` / `apply`; `inject` 的服务名真实存在。
- [] `Config` schema 通过校验; `apply` 拿到的 config 良构。
- [] 工具类: `output.schema` + `render` 完整, `execute` 转发 `exec.signal`。
- [] 副作用走 `ctx.fs` / `ctx.credentials`, 不硬编码秘密。
- [] 资源清理交给 `ctx.effect` (定时器/监听/watcher)。
- [] 跨会话状态按 `session/agent` 键控。
- [] 模型可见表面 (描述/schema/提示词段) 确定性、稳定, 避免无谓的 KV Cache 失效。

## 12.7 一个「从零到运行」的收尾案例

把全书串起来，假设你要做一个「**当前时间工具**」插件并装进 web profile：

```
my-time-tool/  
├─ package.json  
├─ cordis.patch.yml  
└─ lib/  
    └─ index.js
```

```
// lib/index.js
import z from '@deepseek-ai/schemastery'
import { defineTool } from '@deepseek-ai/dsh-tools'

export const name = 'time-tool'
export const inject = ['tools']
export const Config = z.object({ timezone: z.string().default('Asia/Shanghai') })

export function apply(ctx, config) {
  ctx.tools.register(defineTool({
    name: 'current_time',
    description: 'Return the current time in the configured timezone.',
    parameters: {},
    output: {
      schema: {
        type: 'object',
        additionalProperties: false,
        properties: {
          iso: { type: 'string', required: true }, // UTC 基准
          tz: { type: 'string', required: true },
          local: { type: 'string', required: true }, // 配置时区的本地时间
        },
      },
    },
    render: (_a, v) => [{ type: 'text', text: `Now (${v.tz}): ${v.local} (UTC ${v.iso})` }],
    execute() {
      const now = new Date()
      const local = new Intl.DateTimeFormat('en-CA', {
        timeZone: config.timezone,
        year: 'numeric', month: '2-digit', day: '2-digit',
        hour: '2-digit', minute: '2-digit', second: '2-digit', hour12: false,
      }).format(now)
      return { iso: now.toISOString(), tz: config.timezone, local }
    },
  }))
}
```

```
# cordis.patch.yml
- insert:
  - id: time-tool
    name: '@you/my-time-tool'
    config: { timezone: 'Asia/Shanghai' }
```

```
# 安装
dsh plugin --profile web add @you/my-time-tool
# 验证
dsh --profile web --dump-config | grep -A4 time-tool
# 启动
dsh web
```

然后让模型「现在几点」，它就会调用 `current_time`。

---

## 12.8 本章小结

- `--dump-config` / `--dump-default-config` 检查组合树。
- 插件没生效：先看激活（依赖）与 error 态，再看日志。
- patch 层热更新；改源码要重建 + 重启，客户端 HMR 需 `dev:web` 在跑。
- 发布：标准 npm 包；bundle 声明 `dsh.bundle.patch`；客户端声明 `dsh.client`。
- git 插件注意 `allowBuilds`。

至此，全书正文结束。附录 A、B 是速查表。

# 附录 A Cordis API 速查

本节是 `@deepseek-ai/cordis@4.0.1` 的常用 API 速查，供开发时随手翻阅。完整契约以源码类型定义为准。

## A.1 Context

```
import { Context, Service, Inject } from '@deepseek-ai/cordis'

const root = new Context()           // 根容器
root.plugin(plugin, config?)         // 启动插件，返回 Fiber（await 等待启动完成）
root.inject(inject, callback)        // 以依赖 + 回调形式启动插件
root.provide(name, value, check?)    // 提供服务，返回 disposer
root.get(name)                       // 软读取服务（未提供返回 undefined）
root.effect(execute, label?)         // 登记 effect（资源所有权），返回 disposer
root.extend(meta)                    // 派生子作用域（不修改父）
root.isolate(name, label?)           // 为服务名开辟独立作用域
root.intercept(name, config)         // 为服务附加 intercept 配置
```

## A.2 插件形态

```
// 函数插件
export const name = 'x'
export const inject = ['a', 'b']           // 或 { a: config, b: null }
export const Config = z.object({ ... })    // Schemastery schema
export function apply(ctx, config) { ... }

// 类插件
class P { static inject = [...]; constructor(ctx, config) { ... } }

// 对象插件
const p = { name, inject, Config, apply(ctx, config) { ... } }

// 装饰器
@Inject('logger') class S extends Service { ... } // 类依赖
```

## A.3 Service

```
class MyService extends Service {
  static provide = 'myService' // 可选：默认服务名
  constructor(ctx) { super(ctx, 'myService') } // 立即 ctx.provide
}
```

`Service` 关键静态符号：`init`（构造后运行的方法）、`check`（可用性谓词）、`invoke`（可调用服务）、`extend`、`resolveConfig`。

## A.4 事件

```
ctx.on(name, listener) // 注册（随 fiber 注销）；返回 disposer
ctx.once(name, listener) // 一次
ctx.emit(...args) // 同步并发，不等 Promise
ctx.parallel(...args) // 并发并等待全部（收集 rejection 为 AggregateError）
ctx.serial(...args) // 顺序 await，直到 bail 值
ctx.bail(...args) // 同步顺序，直到 bail 值
ctx.waterfall(...args) // 顺序执行，末参数为 next；可拦截/改写/veto
```

「bail」判定：返回值不是 `null` / `false` / `undefined` 即命中停止。

## A.5 生命周期状态（Fiber）

| state | 含义 | |---|---| 0 inactive | 依赖未满足 / 已卸载 | | 1 loading | `_reload` 执行中 | | 2 active | `apply` 已完成 | | 3 error | 启动失败 | | 4 disposed | 已 dispose | | 5 unloading | 正在卸载 |

```
await fiber.await() // 等生命周期稳定；有启动错误则抛
await fiber.dispose() // 卸载（逆序清理 effects）
await fiber.restart() // 卸载后按当前配置重载
await fiber.update(cfg) // 校验新配置 → internal/update waterfall → restart
fiber.getEffects() // 当前 effect 树（诊断）
```

## A.6 错误



| 错误/类 | 含义 | |---|---| | `ValidationError` | 配置 schema 校验失败 ( `invalid config:` + 每条 issue 路径) | | `CordisError('INACTIVE_EFFECT')` | 在已 dispose 的 fiber 上创建 effect | | `service "x" has been registered at <fiber>` | 同名服务重复提供 | | `cannot get required service "x" in inactive context` | 访问未注入/不可用服务 |

## A.7 作用域符号

```
Context.is(value)           // 判断是否为 Cordis context (跨 realm 安全)
Context.filter              // 事件监听作用域过滤 (Symbol)
Context.isolate             // 隔离映射 (Symbol)
Context.intercept           // intercept 配置 (Symbol)
Context.effect              // effect 诊断元数据 (Symbol)
```

## 附录 B DSH 服务目录 ( ctx.\* )

这一节列出插件开发最常打交道的 ctx.\* 服务。它们由 dsh-base / dsh-web-app 等 bundle 提供，宿主平面（进程级）。完整清单见 dsh-tool-cordis 的 API 目录（ cordis\_inspect 工具）。

### B.1 注册表与执行

| 服务 | 键 | 提供方 | 插件作者用途 | |---|---|---|---| | 工具注册表 | ctx.tools | dsh-tools | register(defineTool(...))、guard、schemas || 命令注册表 | ctx.commands | dsh-commands | register({name,description,handler,...}) || Skill 注册表 | ctx.skills | dsh-skill | registerProvider(...) || 系统提示词 | ctx.systemPrompt | dsh-system-prompt | section / context / variable / tools || Agent Preset | ctx.agentPresets | dsh-agent-presets | mount / composeFrom / list / copy |

### B.2 Agent 与会话

| 服务 | 键 | 用途 | |---|---|---| | Agent 工厂 | ctx.agentLoop / ctx.agents | 创建/恢复 agent; setFactory 注册工厂 || 会话 | ctx.session | 持久化日志（append(type, data)） || 会话投影 | ctx.sessionProjections | 从事件推导 UI 状态 (todo 列表等) | | 目标 | ctx.goals | get/create/edit/pause/resume/clear || 子代理 | ctx.subagents | 子代理注册表 |

### B.3 安全与副作用

| 服务 | 键 | 用途 | |---|---|---| | 文件系统 | ctx.fs | resolve / readText / writeText / editText / listDir （走沙箱） || 审批 | ctx.approval | request() （ask/never/allowed-once） || 凭据 | ctx.credentials | resolve / set / unset / describe || 沙箱 | ctx.sandbox | 沙箱策略/执行环境 |

## B.4 模型与网络

| 服务 | 键 | 用途 | |---|---|---| | 模型路由 | `ctx.llm` | 发起模型请求;  
`createUserMessage` 等 | | 默认模型 | `ctx.agentDefaultModel` |  
`currentSelection()` / `saveSelection()` | | 网页检索 | `ctx.web` | 搜索/抓取 |

## B.5 横切与基础设施

| 服务 | 键 | 用途 | |---|---|---| | 压缩 | `ctx.compaction` | `compactIfNeeded` / `compactNow`  
| | Token 计量 | `ctx.tokenMeter` | 上下文 token 统计 | | 后台任务 | `ctx.jobs` | 后台任务  
注册表 | | 定时器 | `ctx.timer` | disposal-aware 定时器 ( `cordis-plugin-timer` ) | | 客  
户端模块 | `ctx.clientModules` | Web 插件表 ( `graph()` / `onRebuilt` ) | | 代码运行时 |  
`ctx.codeRuntime` | `run(request)` (Code Mode 执行后端) |

## B.6 消费方式对照

| 场景 | 用法 | |---|---| | 必须有才能工作 | `export const inject = ['tools']` | | 可选、  
缺失时降级 | `const approval = ctx.get('approval')` | | 按 agent 键控读状态 | 通过  
`exec.agent` / `invocation.agent` 拿到 agent, 再访问其 session | | 惰性读服务实例 |  
`ctx.get('fs')` 返回 `undefined` 若未提供 |

---

本书到此结束。祝你在 DeepSeek Harness 上构建出好用的插件。